

C++从零到高级

语法、资源管理、泛型、并发、性能与工程实践

前言

很多人学 C++ 的痛苦不在语法本身，而在语法背后的执行模型没有连起来：变量到底放在哪里，引用为什么会悬空，构造函数什么时候运行，模板报错为什么像墙一样长，标准库容器为什么有时快、有时慢。只背语法表，很快就会写出能编译但不可维护的程序；只看项目，又会被概念淹没。

这本书按另一条路写：每章先从一个具体小问题开始，再把代码、内存、类型、接口和工程约束一起讲清楚。它不是语法速查表，也不是只讲高级技巧的炫技书。它的目标是让读者从第一段程序走到能写可测试、可维护、可解释性能的现代 C++ 代码。

书中代码默认使用 C++20。示例会优先使用标准库、RAII、值语义、明确所有权和小接口；类成员函数中的成员访问会写出 `this->`，这是为了在教学阶段把“成员”和“局部变量”区分清楚。运行时代码尽量避免递归，树和图的处理会用显式栈、队列或工作表说明，因为工程代码里递归深度常常不是一个可以随手忽略的细节。

核心思想

学 C++ 不是记住更多语法，而是建立三条主线：第一，值和对象如何存在；第二，资源如何被拥有和释放；第三，接口如何把复杂性限制在可测试的边界内。

如何阅读本书

如果你刚开始学 C++，可以按顺序读第一到第七章。不要急着跳到模板、并发和性能；如果你还不能解释一个对象何时构造、何时析构、谁拥有资源，那么高级语法只会放大错误。

如果你已经会写基本程序，可以重点读第五章到第十三章。读的时候建议做三件事：第一，手敲每章的核心代码；第二，故意改坏一个边界条件，看编译器、测试或运行时如何反馈；第三，把本章的接口改成自己的小工具，而不是只复制书里的函数。

每章会反复出现几个固定栏目：

- **问题入口**：先给一个真实的小需求。
- **朴素写法**：先写能想到的直接方案。
- **失败原因**：解释它为什么会错、慢或难维护。
- **现代写法**：用标准库和语言机制修正。
- **边界检查**：列出容易遗漏的输入、生命周期和复杂度。

常见误区

不要把书中代码当成唯一答案。真正要学的是代码背后的不变量：哪些数据必须一直成立，哪些资源只有一个拥有者，哪些函数只观察不修改，哪些错误必须显式返回或抛出。

目录

第一部分	从第一段程序到可解释的执行	
第 1 章	先建立执行模型	2
第 2 章	第一段可维护程序	6
第二部分	语言基础：值、控制流、函数和抽象	
第 3 章	值、类型和初始化	12
第 4 章	控制流、函数和错误路径	16
第 5 章	类、封装和不变量	20
第三部分	现代 Cpp 的核心：资源、对象和容器	
第 6 章	生命周期、所有权和 RAII	25
第 7 章	容器、迭代器和标准库思维	31
第 8 章	错误处理和接口设计	35
第四部分	泛型、标准库和可组合设计	
第 9 章	模板、concept 和泛型设计	40
第 10 章	标准算法、ranges 和可组合代码	45
第五部分	工程化：构建、测试、调试和性能	
第 11 章	构建、测试和调试	50
第 12 章	性能、内存和测量	54
第六部分	高级专题：并发、异步和大型代码组织	
第 13 章	并发、线程和同步	59
第 14 章	大型代码组织和演进	64
第七部分	综合项目：把语言能力落到工程	
第 15 章	项目一：命令行词频统计器	69
第 16 章	项目二：实现一个 LRU 缓存	75
附录 A	学习检查清单	81
附录 B	标准库能力地图	82
术语表		84

第零部分

从第一段程序到可解释的执行

第 1 章

先建立执行模型

1.1 从一个错误的猜测开始

很多入门材料会直接讲 `int`、`if`、`for`、`class`。这些概念当然重要，但如果一开始不回答“程序运行时到底发生了什么”，后面的指针、引用、移动语义和 RAII 都会像孤立规则。我们先看一个最小程序：

Listing 1.1 一个看似普通的程序

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
6     std::string name = "Ada";
7     int score = 95;
8     std::cout << name << " " << score << '\n';
9     return 0;
10 }
```

这段程序能输出 `Ada 95`。如果只从语法角度看，它包含头文件、变量、输出和返回值。但从执行模型看，它还包含更多信息：`name` 是一个对象，它管理一段字符资源；`score` 是一个整数值；`std::cout` 是标准库提供的输出流；程序从 `main` 开始执行，函数结束时局部对象按逆序销毁。

心智模型

C++ 程序可以先用四个问题观察：对象在哪里存在，谁拥有资源，什么时候构造和析构，错误如何被发现和传播。语法只是表达这些事实的工具。

1.2 编译不是运行

初学者常把“能编译”理解成“程序正确”。这不够。编译器主要检查语法、类型和一部分静态规则；运行时才会遇到输入、文件、内存、时间和并发调度。下面的程序能编译，但它不可靠：

Listing 1.2 能编译但会越界的程序

```
1 #include <iostream>
2 #include <vector>
3
4 int main()
5 {
6     std::vector<int> scores{90, 85, 77};
7     std::cout << scores[3] << '\n';
8 }
```

`scores[3]` 访问第四个元素，但数组只有三个元素。标准库的 `operator[]` 通常不会做边界检查，程序可能输出杂乱值，也可能崩溃。改成 `scores.at(3)` 后，标准库会在越界时抛出异常。这里不是说所有代码都必须用 `at`，而是要理解：接口选择影响错误出现的位置。

常见误区

能编译只说明程序通过了语言规则检查，不说明数据范围、资源生命周期和业务不变量正确。写 C++ 时必须同时关心编译期和运行期。

1.3 从源码到可执行文件

一个常见构建链条是：预处理、编译、汇编、链接。预处理处理 `##include` 和宏；编译把翻译单元转换成目标代码；链接把多个目标文件和库合成可执行文件。先记住这个模型，后面遇到“头文件重复定义”“链接找不到符号”“模板为什么要放头文件里”时就不会只靠猜。

Listing 1.3 手动观察构建步骤

```
1 g++ -std=c++20 -E main.cpp -o main.i
2 g++ -std=c++20 -S main.cpp -o main.s
3 g++ -std=c++20 -c main.cpp -o main.o
4 g++ main.o -o app
```

第一步生成的 `main.i` 会很大，因为头文件内容被展开进来。第三步生成 `main.o`，它还不是最终程序。第四步链接时，链接器需要找到 `std::cout`、字符串构造函数等符号的实现。

1.4 对象、值和名字

变量名不是对象本身，而是访问对象的名字。下面的代码里，`x` 和 `y` 是两个不同对象：

Listing 1.4 复制会产生新对象

```
1 int x = 10;
2 int y = x;
3 y = 20;
```

执行后 `x` 仍然是 10，`y` 变成 20。如果换成引用，情况就不同：

Listing 1.5 引用是已有对象的别名

```
1 int x = 10;
2 int& alias = x;
3 alias = 20;
```

这时 `x` 也变成 20。学习 C++ 的第一个分水岭就在这里：你必须看清一行代码是在创建新对象、观察旧对象，还是通过别名修改旧对象。

核心思想

名字、对象和值要分开理解。名字是访问入口，对象有生命周期，值是对象当前保存的信息。引用不创建新对象，它让一个已有对象有了另一个访问入口。

1.5 本章边界检查

读完本章后，你应该能回答这些问题：

- `std::string name = "Ada"` 创建了什么对象？
- 为什么 `scores[3]` 能编译却不可靠？
- 编译错误和链接错误有什么区别？
- 复制和引用在对象层面有什么不同？

后面的章节会不断回到这些问题。只要执行模型稳，语法规则就不再是散点。

1.6 把一程序拆成执行步骤

为了让这个模型不只停留在口号上，我们把第一段程序里的两行拆开看：

Listing 1.6 两行代码里的对象变化

```
1 std::string name = "Ada";
2 std::cout << name << '\n';
```

第一行不是“变量等于字符串”这么简单。更准确的过程是：编译器看到左边类型是 `std::string`，右边是字符串字面量；于是构造一个 `std::string` 对象，让这个对象保存字符内容，并把名字 `name` 绑定到这个对象。第二行也不是把对象复制到屏幕上，而是调用输出流的插入操作，把 `name` 当前表示的字符序列写到标准输出。

如果把 `name` 改成整数，输出语句仍然长得差不多：

Listing 1.7 同一个输出符号背后是不同操作

```
1 int score = 95;
2 std::cout << score << '\n';
```

这说明 `<<` 不是一个固定的机器动作，而是一组根据类型选择的操作。初学时不用急着理解“重载”的全部规则，但要先记住：C++ 代码表面相似，不代表执行细节相同。类型决定能做什么，也决定怎么做。

1.7 三类错误要分开定位

写程序时常见三类错误。第一类是编译错误，例如少分号、类型不匹配、调用不存在的函数。第二类是链接错误，例如声明了函数却没有提供实现。第三类是运行错误，例如越界、除零、文件不存在、输入格式不对。

Listing 1.8 声明了函数但没有实现会导致链接错误

```
1 int add(int lhs, int rhs);
2
3 int main()
4 {
5     return add(1, 2);
6 }
```

这段代码能通过语法检查，因为编译器知道 `add` 的参数和返回类型。但链接时找不到函数体，所以会失败。许多初学者看到一屏英文报错就慌，其实只要先分类，定位就清楚很多：编译错看当前源文件和类型，链接错看函数实现和目标文件，运行错看输入、数据范围和生命周期。

1.8 一个最小实验流程

建议从第一章开始就固定一个实验流程。把下面代码保存为 `main.cpp`：

Listing 1.9 最小实验程序

```
1 #include <iostream>
2
3 int main()
4 {
5     int value = 41;
6     ++value;
7     std::cout << value << '\n';
```



```
8 }
```

用下面命令构建并运行：

Listing 1.10 手动编译运行

```
1 g++ -std=c++20 -Wall -Wextra -pedantic main.cpp -o app
2 ./app
```

`-Wall -Wextra -pedantic` 会打开更多警告。警告不是噪声，它经常指出“能编译但值得怀疑”的代码。以后每写一个小程序，都先做到三件事：能用固定命令构建，能说明输出为什么是这个值，能解释如果输入为空或越界会怎样。

习题与作业

把 `scores[3]` 改成 `scores.at(3)`，重新运行并观察错误出现在哪里。再把 `std::string name = "Ada"` 改成 `auto name = "Ada"`，思考这时 `name` 的类型是不是 `std::string`。这两个小实验分别训练运行期边界和类型推导。

第 2 章

第一段可维护程序

2.1 问题入口：统计一组成绩

第一段程序不要只写 `Hello, world`。我们从小需求开始：读入若干成绩，输出人数、总分、平均分、最高分和最低分。这个需求足够小，但已经覆盖输入、容器、循环、函数和错误处理。最直接的写法可能是把所有逻辑塞进 `main`：

Listing 2.1 所有逻辑挤在 `main` 里

```
1 #include <iostream>
2 #include <vector>
3
4 int main()
5 {
6     std::vector<int> scores;
7     int value = 0;
8     while (std::cin >> value) {
9         scores.push_back(value);
10    }
11
12    int sum = 0;
13    int min_score = scores[0];
14    int max_score = scores[0];
15    for (int score : scores) {
16        sum += score;
17        if (score < min_score) {
18            min_score = score;
19        }
20        if (score > max_score) {
21            max_score = score;
22        }
23    }
24
25    std::cout << scores.size() << '\n';
26    std::cout << sum / scores.size() << '\n';
27    std::cout << min_score << " " << max_score << '\n';
28 }
```

它能在普通输入下运行，但有三个问题。第一，空输入时 `scores[0]` 越界。第二，平均分用了整数除法，95 和 96 的平均值会被截断。第三，读取、统计和输出混在一起，后面不好测试。

2.2 把数据结果变成类型

先设计一个结果类型。统计结果不是五个散落的变量，而是一个概念：`ScoreSummary`。

Listing 2.2 用结构体表达统计结果

```
1 #include <cstdint>
2
```

```

3 struct ScoreSummary {
4     std::size_t count = 0;
5     int sum = 0;
6     int min_score = 0;
7     int max_score = 0;
8     double average = 0.0;
9 };

```

这个结构体没有行为，只是承载结果。它的默认值也有意义：当没有成绩时，`count` 是 0，平均值是 0.0。但是否允许空输入，不应该靠默认值偷偷决定，而应该由统计函数明确处理。

2.3 函数边界：只观察，不拥有

统计函数只需要观察一段成绩，不需要复制它。用 `std::span<const int>` 可以表达“我看一段连续整数，但不拥有它”。

Listing 2.3 统计函数使用 `span` 观察数据

```

1 #include <algorithm>
2 #include <numeric>
3 #include <optional>
4 #include <span>
5 #include <vector>
6
7 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores)
8 {
9     if (scores.empty()) {
10         return std::nullopt;
11     }
12
13     const auto [min_it, max_it] = std::minmax_element(scores.begin(), scores.end());
14     const int sum = std::accumulate(scores.begin(), scores.end(), 0);
15
16     ScoreSummary summary;
17     summary.count = scores.size();
18     summary.sum = sum;
19     summary.min_score = *min_it;
20     summary.max_score = *max_it;
21     summary.average = static_cast<double>(sum) / static_cast<double>(scores.size());
22     return summary;
23 }

```

`std::optional` 表达“可能没有结果”。这比返回一个看似正常但其实无意义的 `ScoreSummary` 更诚实。调用者必须处理空输入。

2.4 输入和输出留在边缘

现在 `main` 只做三件事：读入、调用核心逻辑、输出结果。

Listing 2.4 把核心逻辑从 `main` 中拆出来

```

1 #include <iostream>
2
3 std::vector<int> read_scores()
4 {

```

```

5     std::vector<int> scores;
6     int value = 0;
7     while (std::cin >> value) {
8         scores.push_back(value);
9     }
10    return scores;
11 }
12
13 int main()
14 {
15     const std::vector<int> scores = read_scores();
16     const std::optional<ScoreSummary> summary = summarize_scores(scores);
17     if (!summary.has_value()) {
18         std::cerr << "no scores\n";
19         return 1;
20     }
21
22     std::cout << "count: " << summary->count << '\n';
23     std::cout << "sum: " << summary->sum << '\n';
24     std::cout << "average: " << summary->average << '\n';
25     std::cout << "range: " << summary->min_score
26                 << "..." << summary->max_score << '\n';
27 }

```

这段代码的重点不是更长，而是责任更清楚。读输入的函数可以单独替换，统计函数可以用固定数组测试，输出格式可以改而不碰统计逻辑。

核心思想

好程序不是从“少写几行”开始，而是从“每段代码只负责一件事”开始。核心逻辑尽量不直接依赖终端、文件和全局状态。

2.5 本章边界检查

这个小程序至少要检查四类输入：

- 空输入：应返回 `std::nullopt`。
- 单个成绩：最小值和最大值相同。
- 多个成绩：平均分不能被整数除法截断。
- 非法文本：当前读入逻辑会在失败处停止，这是否符合需求要由调用场景决定。

从第一段程序开始，就把“能跑”和“可解释、可测试、可维护”分开。后者才是工程里的 C++。

2.6 一步一步推导统计函数

为什么统计函数要先处理空输入？从最小反例看最清楚。输入为空时，根本不存在最小值、最大值和平均值。如果代码为了省事把它们设成 `0`，调用者就会误以为真的统计到了一个值为 `0` 的成绩。于是函数的返回类型不能只是 `ScoreSummary`，还要能表达“没有结果”。

再看单元素输入：

Listing 2.5 单元素时每个字段都能推导出来

```

1 const std::vector<int> scores{91};
2 // count = 1
3 // sum = 91
4 // min_score = 91

```

```
5 // max_score = 91
6 // average = 91.0
```

单元素不是特殊麻烦，而是最好的基准。只要它成立，多元素就可以理解成“把同一套规则重复应用”。扫描第一个元素时，最小值和最大值都等于它；扫描后续元素时，只在发现更小或更大时更新边界。标准库的 `std::minmax_element` 把这个扫描过程封装好了，但你仍然要知道它在做什么。

2.7 完整文件版本

前面的代码片段可以合成一个可运行文件。入门阶段不要只看片段，至少要能把它们合起来编译：

Listing 2.6 完整成绩统计程序

```
1 #include <algorithm>
2 #include <iostream>
3 #include <numeric>
4 #include <optional>
5 #include <span>
6 #include <vector>
7
8 struct ScoreSummary {
9     std::size_t count = 0;
10    int sum = 0;
11    int min_score = 0;
12    int max_score = 0;
13    double average = 0.0;
14 };
15
16 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores)
17 {
18     if (scores.empty()) {
19         return std::nullopt;
20     }
21
22     const auto [min_it, max_it] = std::minmax_element(scores.begin(), scores.end());
23     const int sum = std::accumulate(scores.begin(), scores.end(), 0);
24
25     return ScoreSummary{
26         .count = scores.size(),
27         .sum = sum,
28         .min_score = *min_it,
29         .max_score = *max_it,
30         .average = static_cast<double>(sum) / static_cast<double>(scores.size()),
31     };
32 }
33
34 std::vector<int> read_scores()
35 {
36     std::vector<int> scores;
37     int value = 0;
38     while (std::cin >> value) {
39         scores.push_back(value);
```

```

40     }
41     return scores;
42 }
43
44 int main()
45 {
46     const std::vector<int> scores = read_scores();
47     const std::optional<ScoreSummary> summary = summarize_scores(scores);
48     if (!summary.has_value()) {
49         std::cerr << "no scores\n";
50         return 1;
51     }
52
53     std::cout << "count: " << summary->count << '\n';
54     std::cout << "sum: " << summary->sum << '\n';
55     std::cout << "average: " << summary->average << '\n';
56     std::cout << "range: " << summary->min_score
57         << "..." << summary->max_score << '\n';
58 }

```

运行时可以这样喂输入：

Listing 2.7 用管道提供输入

```
1 echo "90 80 100" | ./score
```

如果没有任何输入，程序返回错误码 **1**。如果有输入，核心统计逻辑完全不关心输入来自键盘、文件还是测试数组。这就是把边界留在边缘的价值。

2.8 从测试反推接口

接口不是凭感觉定的。先写出想验证的行为：

Listing 2.8 从行为反推接口

```

1 const std::vector<int> empty;
2 assert(!summarize_scores(empty).has_value());
3
4 const std::vector<int> one{88};
5 const auto single = summarize_scores(one);
6 assert(single.has_value());
7 assert(single->min_score == 88);
8 assert(single->max_score == 88);
9 assert(single->average == 88.0);

```

这些测试逼着我们承认两个事实：空输入没有统计结果，非空输入必须保留小数平均值。于是 `std::optional<ScoreSummary>` 和 `double average` 都不是装饰，而是从需求和边界推出来的。

习题与作业

把成绩限制为 **0** 到 **100**。不要在统计函数里偷偷忽略非法值，而是设计一个读取或校验阶段，让非法输入能被明确报告。完成后再给空输入、负数、超过 **100**、单元素和普通多元素各写一个测试。

第零部分

语言基础：值、控制流、函数和抽象

第 3 章

值、类型和初始化

3.1 问题入口：金额为什么不能随使用 double

假设要写一个购物车，最直接的金额表示是 `double`：

Listing 3.1 金额直接用 `double` 的朴素写法

```
1 double price = 0.1;
2 double total = price + price + price;
3 if (total == 0.3) {
4     // ...
5 }
```

这段代码的问题不在 C++ 独有，而在二进制浮点无法精确表示很多十进制小数。 $0.1 + 0.1 + 0.1$ 未必和 0.3 完全相等。金额通常应该用最小单位的整数，例如“分”。

Listing 3.2 用整数分表示金额

```
1 #include <stdint>
2
3 struct Money {
4     std::int64_t cents = 0;
5 };
6
7 Money add(Money lhs, Money rhs)
8 {
9     return Money{.cents = lhs.cents + rhs.cents};
10 }
```

这里的重点是类型不是装饰。类型表达业务含义，也限制错误。一个普通 `int` 可能是年龄、下标、数量、分数或错误码；而 `Money` 明确告诉读者它表示金额。

3.2 初始化：少给错误留入口

C++ 有多种初始化形式。入门阶段建议优先使用大括号初始化，因为它能挡住一部分窄化转换。

Listing 3.3 大括号初始化拒绝窄化转换

```
1 int ok{42};
2 // int bad{3.14}; // 编译错误：double 到 int 会丢失信息
3 int truncated = 3.14; // 可以编译，但值变成 3
```

不要把这理解成“永远只用一种形式”。构造对象、调用工厂函数、使用标准库容器时会遇到不同写法。先建立原则：初始化应该让对象一出生就处在有效状态。

3.3 `const` 不是形式主义

`const` 的意义是把“不修改”写进接口。下面这个函数只观察价格列表，不应该复制，也不应该修改。

Listing 3.4 用 const 引用表达只读访问

```

1 #include <vector>
2
3 std::int64_t total_cents(const std::vector<Money>& items)
4 {
5     std::int64_t total = 0;
6     for (const Money& item : items) {
7         total += item.cents;
8     }
9     return total;
10 }

```

如果写成 `std::vector<Money> items`，每次调用都会复制整个数组。复制不一定错误，但这里没有必要。接口应该表达意图：观察就用 `const&` 或 `std::span<const T>`，拥有或需要本地副本时才按值传递。

3.4 auto 的正确用法

`auto` 可以减少重复类型，但不能掩盖所有权和复制。

Listing 3.5 range-for 中避免不必要复制

```

1 std::vector<std::string> names = {"Ada", "Bjarne", "Grace"};
2
3 for (const auto& name : names) {
4     // 只观察，不复制 string
5 }
6
7 for (auto name : names) {
8     // 每轮复制一个 string，除非确实需要本地副本
9 }

```

教学阶段要养成一个习惯：看到 `auto`，仍然要问它推导出的到底是值、引用还是指针。语法短不等于语义简单。

3.5 边界：整数溢出和符号混用

整数类型也有坑。`int` 通常是 32 位，累加大数据时可能溢出。`std::size_t` 是无符号类型，和 `int` 混用时可能产生意外比较。

Listing 3.6 避免有符号和无符号混乱

```

1 std::vector<int> values{1, 2, 3};
2 for (std::size_t i = 0; i < values.size(); ++i) {
3     // i 和 size() 类型一致
4 }

```

但如果下标要向左移动到 `-1` 表示结束，`std::size_t` 就不适合。类型选择不是背规则，而是看值域和操作。

常见误区

类型错误常常不是编译错误，而是建模错误。金额、长度、数量、索引、标识符和错误码不要长期混在同一种裸整数里。

3.6 从金额例子继续推到类型设计

只把金额改成整数分还不够。下面这段代码仍然可能把“分”和“件数”混起来：

Listing 3.7 裸整数仍然容易混淆含义

```
1 std::int64_t cents = 1299;
2 int quantity = 3;
3 std::int64_t wrong = cents + quantity; // 编译器不知道业务上不合理
```

把金额包装成 **Money** 后，错误会更难发生。再进一步，可以把常见操作写成函数：

Listing 3.8 让金额操作表达业务含义

```
1 struct Money {
2     std::int64_t cents = 0;
3 };
4
5 Money multiply(Money price, int quantity)
6 {
7     return Money{.cents = price.cents * quantity};
8 }
9
10 Money add(Money lhs, Money rhs)
11 {
12     return Money{.cents = lhs.cents + rhs.cents};
13 }
```

这不是为了让代码变繁琐，而是把业务边界交给类型系统。以后如果要禁止负金额、处理货币种类、做舍入策略，修改点会集中在 **Money** 附近，而不是散落在所有 **int** 计算里。

3.7 值、引用和视图的选择

函数参数最常见的三种形态是按值、按引用和视图。可以用一个用户列表作为判断样本：

Listing 3.9 三种参数形态表达不同意图

```
1 struct User {
2     int id = 0;
3     std::string name;
4 };
5
6 void rename(User& user, std::string new_name)
7 {
8     user.name = std::move(new_name);
9 }
10
11 void print_user(const User& user)
12 {
13     std::cout << user.id << " " << user.name << '\n';
14 }
15
16 std::size_t total_name_length(std::span<const User> users)
17 {
18     std::size_t total = 0;
19     for (const User& user : users) {
```

```

20     total += user.name.size();
21 }
22 return total;
23 }

```

`rename` 要修改已有对象，所以用非 `const` 引用。`print_user` 只观察一个对象，所以用 `const&`。`total_name_length` 观察一段连续对象，所以用 `std::span<const User>`。这三个接口的区别不是语法偏好，而是所有权和修改权限不同。

3.8 初始化和有效状态

初始化的目标是让对象一出生就可用。反例是先创建半成品，再靠后续赋值补齐：

Listing 3.10 半成品对象容易被提前使用

```

1 Money price;
2 // 中间几十行代码
3 price.cents = 1299;

```

更好的写法是直接给出有效值：

Listing 3.11 直接构造有效值

```

1 Money price{.cents = 1299};

```

如果值需要校验，就不要暴露裸字段，而是放到类的构造函数或工厂函数里。也就是说，初始化不是“把值塞进去”，而是“建立一个满足规则的对象”。

习题与作业

设计一个 `Percent` 类型表示折扣百分比，要求范围是 `0` 到 `100`。先写一个能被误用的裸 `int` 版本，再把它改成类型。重点观察：哪些错误从运行期判断提前到了构造边界。

第 4 章

控制流、函数和错误路径

4.1 问题入口：命令行参数解析

控制流不是只会写 `if` 和 `for`。真正的训练是把正常路径和错误路径写清楚。假设程序支持：

Listing 4.1 命令行需求

```
1 tool --input data.txt --limit 100
```

朴素写法会在 `main` 里一边循环一边判断：

Listing 4.2 混乱的参数解析

```
1 for (int i = 1; i < argc; ++i) {
2     std::string arg = argv[i];
3     if (arg == "--input") {
4         input = argv[++i];
5     } else if (arg == "--limit") {
6         limit = std::stoi(argv[++i]);
7     }
8 }
```

这段代码漏了很多边界：`-input` 后面没有值会越界，`-limit abc` 会抛异常，重复参数如何处理也不明确。

4.2 先定义结果

参数解析的结果应该是一个类型：

Listing 4.3 参数解析结果

```
1 #include <optional>
2 #include <string>
3
4 struct Options {
5     std::string input_path;
6     int limit = 0;
7 };
8
9 struct ParseError {
10     std::string message;
11 };
```

标准库直到 C++23 才有 `std::expected`。为了保持 C++20，本章用一个简单结构表达“成功或失败”：

Listing 4.4 C++20 下的简单结果类型

```
1 struct ParseResult {
2     std::optional<Options> options;
```

```

3     std::optional<ParseError> error;
4 };

```

真实项目可以使用成熟的 `expected` 实现；教材这里先把控制流讲清楚。

4.3 让循环只做推进

解析函数可以按下标推进。每次遇到需要值的选项，都先检查后面是否还有参数。

Listing 4.5 明确错误路径的参数解析

```

1 #include <cstdlib>
2 #include <string_view>
3 #include <vector>
4
5 ParseResult parse_options(std::span<char* const> args)
6 {
7     Options options;
8     bool has_input = false;
9     bool has_limit = false;
10
11     for (std::size_t i = 1; i < args.size(); ++i) {
12         const std::string_view arg = args[i];
13         if (arg == "--input") {
14             if (i + 1 >= args.size()) {
15                 return {.error = ParseError{"--input requires a value"}};
16             }
17             options.input_path = args[++i];
18             has_input = true;
19             continue;
20         }
21         if (arg == "--limit") {
22             if (i + 1 >= args.size()) {
23                 return {.error = ParseError{"--limit requires a value"}};
24             }
25             const std::string value = args[++i];
26             try {
27                 options.limit = std::stoi(value);
28             } catch (const std::exception&) {
29                 return {.error = ParseError{"--limit must be an integer"}};
30             }
31             has_limit = true;
32             continue;
33         }
34         return {.error = ParseError{"unknown option"}};
35     }
36
37     if (!has_input) {
38         return {.error = ParseError{"missing --input"}};
39     }
40     if (!has_limit) {
41         return {.error = ParseError{"missing --limit"}};
42     }
43     return {.options = options};

```

```
44 }
```

这段代码不追求最短。它追求每个错误路径都能解释。工程代码里，错误消息是接口的一部分。

4.4 函数大小和责任边界

当解析规则继续增多，可以把“取下一个值”抽成辅助函数，把“字符串转整数”抽成辅助函数。拆函数不是为了好看，而是为了隔离可测试的决策点。

Listing 4.6 把字符串转整数变成独立函数

```
1 std::optional<int> parse_int(std::string_view text)
2 {
3     try {
4         std::size_t consumed = 0;
5         const int value = std::stoi(std::string{text}, &consumed);
6         if (consumed != text.size()) {
7             return std::nullopt;
8         }
9         return value;
10    } catch (const std::exception&) {
11        return std::nullopt;
12    }
13 }
```

这里有一个小成本：`std::stoi` 接受 `std::string`，所以我们从 `string_view` 构造了字符串。性能敏感场景可以用 `std::from_chars`。入门阶段先把正确性和接口讲清楚。

4.5 本章边界检查

控制流代码要重点检查：

- 选项缺值。
- 未知选项。
- 数字格式错误。
- 重复选项是否允许。
- 默认值是否真实表达业务需求。

核心思想

控制流的高级能力不是写复杂分支，而是让每条路径都有清晰含义。正常路径短，错误路径明确，状态推进单调，函数边界可测试。

4.6 从缺值反例推到下标推进规则

参数解析最容易错在 `++i`。看这个输入：

Listing 4.7 缺少参数值的输入

```
1 tool --input
```

如果代码直接写 `input = argv[++i]`，`i` 会越过最后一个参数。这个反例推出第一条规则：任何消耗下一个参数的选项，都必须先检查 `i + 1 < args.size()`。再看另一个输入：

Listing 4.8 未知选项不能静默忽略

```
1 tool --input data.txt --limit 100
```

`-limit` 是拼写错误。如果解析器静默忽略未知选项，程序可能用默认限制继续跑，结果比直接失败更危险。这个反例推出第二条规则：未知选项应该成为错误，除非需求明确允许透传。

4.7 让数字解析没有半截成功

`std::stoi("12abc")` 会解析出 12，如果不检查消费长度，就会把坏输入当成好输入。前面 `parse_int` 里检查 `consumed != text.size()`，就是为了排除这种半截成功。

在 C++20 中也可以用 `std::from_chars`，它不抛异常，也不受本地化影响：

Listing 4.9 用字符转换解析整数

```
1 #include <charconv>
2
3 std::optional<int> parse_int_fast(std::string_view text)
4 {
5     int value = 0;
6     const char* begin = text.data();
7     const char* end = text.data() + text.size();
8     const auto [ptr, ec] = std::from_chars(begin, end, value);
9     if (ec != std::errc{} || ptr != end) {
10         return std::nullopt;
11     }
12     return value;
13 }
```

这个版本更适合底层解析。它的接口仍然返回 `std::optional<int>`，因为调用者关心的是“是否完整解析成功”，而不是底层用了异常还是错误码。

4.8 把主流程写成可读句子

解析函数写好后，`main` 不应该继续塞满细节。入口可以保持很薄：

Listing 4.10 入口只负责连接步骤

```
1 int run(std::span<char* const> args)
2 {
3     const ParseResult parsed = parse_options(args);
4     if (parsed.error.has_value()) {
5         std::cerr << parsed.error->message << '\n';
6         return 2;
7     }
8
9     return run_with_options(*parsed.options);
10 }
```

这里有一个隐含要求：`ParseResult` 不能同时既有 `options` 又有 `error`。更严谨的项目会用 `expected` 或自定义变体类型表达二选一。教材这里先用简单结构，是为了把控制流看清楚；真正工程里可以进一步收紧类型。

习题与作业

给参数解析器补上重复选项规则：`-limit 10 -limit 20` 是报错，还是后者覆盖前者？先写出你的规则，再修改 `has_limit` 分支，让测试覆盖这两种输入。

第 5 章

类、封装和不变量

5.1 问题入口：温度范围

类不是把几个变量包起来。类的价值在于维护不变量。不变量（invariant）指对象在任何对外可见状态下都必须成立的条件。假设要表示一个温度区间，要求最低温不能高于最高温。

朴素结构体写法如下：

Listing 5.1 没有保护不变量的结构体

```
1 struct TemperatureRange {
2     double low = 0.0;
3     double high = 0.0;
4 };
5
6 TemperatureRange range{.low = 30.0, .high = 10.0}; // 逻辑错误
```

编译器不知道业务规则，所以这段代码能编译。要把规则放进类型里。

5.2 构造函数建立有效状态

Listing 5.2 构造函数检查不变量

```
1 #include <stdexcept>
2
3 class TemperatureRange {
4 public:
5     TemperatureRange(double low, double high)
6         : low_{low}, high_{high}
7     {
8         if (high < low) {
9             throw std::invalid_argument{"high must not be lower than low"};
10        }
11    }
12
13    [[nodiscard]] double low() const noexcept
14    {
15        return this->low_;
16    }
17
18    [[nodiscard]] double high() const noexcept
19    {
20        return this->high_;
21    }
22
23    [[nodiscard]] bool contains(double value) const noexcept
24    {
25        return this->low_ <= value && value <= this->high_;
26    }
27 }
```



```

27
28 private:
29     double low_ = 0.0;
30     double high_ = 0.0;
31 };

```

数据成员是私有的，调用者不能绕过构造函数随意改。成员函数里使用 `this->`，目的是把“当前对象的成员”写清楚。

5.3 setter 不一定是封装

很多人把封装理解成“成员私有，再给每个成员写 getter 和 setter”。这只是形式。如果写出下面的接口，不变量又被破坏了：

Listing 5.3 危险的 setter

```

1 void set_low(double value)
2 {
3     this->low_ = value; // 可能让 low_ > high_
4 }

```

更好的接口应该表达业务动作。例如扩展范围：

Listing 5.4 用业务动作维护不变量

```

1 void expand_to_include(double value) noexcept
2 {
3     if (value < this->low_) {
4         this->low_ = value;
5     }
6     if (value > this->high_) {
7         this->high_ = value;
8     }
9 }

```

调用者不需要知道内部如何保存，只知道对象会把某个值纳入范围。

5.4 值语义优先

`TemperatureRange` 可以复制、赋值、放进 `std::vector`。这是值语义。值语义让对象像数字和字符串一样好用。

Listing 5.5 值语义让对象易组合

```

1 std::vector<TemperatureRange> ranges;
2 ranges.emplace_back(10.0, 20.0);
3 ranges.emplace_back(-5.0, 8.0);

```

不要过早把所有类都设计成继承体系。许多业务概念更适合小而明确的值对象。

5.5 本章边界检查

设计类时至少问：

- 对象什么时候有效？
- 哪些状态绝对不允许出现？

- 构造函数能否一次建立有效状态？
- 对外接口是业务动作，还是裸字段修改？
- 对象是否应该支持复制和移动？

核心思想

类的核心不是语法里的 `class`，而是把数据和规则放在同一个边界内。好的类让错误状态难以构造，让正确用法自然出现。

5.6 从坏对象推导构造边界

继续看温度区间。为什么不能让调用者先默认构造，再分别设置上下界？因为中间状态可能已经被观察到：

Listing 5.6 分步设置制造临时坏状态

```
1 TemperatureRange range;
2 range.set_high(10.0);
3 range.set_low(30.0); // 如果允许，就破坏 low <= high
4 if (range.contains(20.0)) {
5     // 这里的判断已经没有可信含义
6 }
```

这个反例推出一条类设计规则：如果一个对象必须同时拥有几项数据才有效，就让构造函数一次收齐这些数据，并在构造边界检查。不应该把“不完整但暂时别用”的对象暴露给普通调用者。

5.7 成员函数应该维护对象语义

假设业务需要把两个温度区间合并。不要把内部字段拿出来让调用者自己算，而是提供表达业务的函数：

Listing 5.7 合并两个有效区间

```
1 TemperatureRange merge(TemperatureRange lhs, TemperatureRange rhs)
2 {
3     const double low = std::min(lhs.low(), rhs.low());
4     const double high = std::max(lhs.high(), rhs.high());
5     return TemperatureRange{low, high};
6 }
```

因为 `lhs` 和 `rhs` 都已经是有有效对象，`min(low)` 和 `max(high)` 构造出来的新对象也应当有效。这里的推理链条很重要：类不变性让函数可以建立在更少的防御性检查上。

5.8 什么时候用 `struct`，什么时候用 `class`

C++ 里 `struct` 和 `class` 的主要语法差异是默认访问权限；真正的设计差异来自意图。只承载数据、没有复杂不变量的聚合，适合 `struct`：

Listing 5.8 简单结果适合 `struct`

```
1 struct ScoreSummary {
2     std::size_t count = 0;
3     int sum = 0;
4     double average = 0.0;
5 };
```

需要保护规则、隐藏表示、控制修改路径的概念，适合 `class`。这不是硬性宗教，而是让读者一看到类型形态，就能大致知道它的使用方式。

5.9 类的测试不是只测 `getter`

测试 `TemperatureRange` 时，不要只检查 `low()` 和 `high()` 返回原值。更重要的是不变量：

Listing 5.9 围绕不变量测试类

```
1 const TemperatureRange normal{10.0, 20.0};
2 assert(normal.contains(10.0));
3 assert(normal.contains(15.0));
4 assert(!normal.contains(21.0));
5
6 bool rejected = false;
7 try {
8     const TemperatureRange invalid{30.0, 10.0};
9 } catch (const std::invalid_argument&) {
10     rejected = true;
11 }
12 assert(rejected);
```

如果一个类的测试只是在验证字段保存和取出，说明这个类可能没有承担真正职责；它也许只是一个 `struct`，或者需要重新思考业务动作。

习题与作业

实现一个 `Date` 类，要求月份在 **1** 到 **12**，日期必须符合月份天数。先不处理闰年，再把闰年规则加进去。练习重点不是背日期算法，而是把“不允许出现的状态”挡在构造函数外。

第零部分

现代 Cpp 的核心：资源、对象和容器

第 6 章

生命周期、所有权和 RAII

6.1 问题入口：文件忘记关闭

资源不只是内存。文件句柄、锁、网络连接、临时目录、线程都属于资源。C 风格写法常见模式是打开后手动关闭：

Listing 6.1 手动管理文件资源容易遗漏

```
1 #include <cstdio>
2
3 void write_message()
4 {
5     FILE* file = std::fopen("out.txt", "w");
6     if (file == nullptr) {
7         return;
8     }
9     std::fprintf(file, "hello\n");
10    std::fclose(file);
11 }
```

这段代码看起来没问题，但一旦中间出现多个返回路径或异常，`fclose` 就容易漏。现代 C++ 的核心习惯是 RAII：资源获取即初始化，对象析构即释放。

6.2 RAII 的基本形状

Listing 6.2 用 `ofstream` 自动管理文件

```
1 #include <fstream>
2
3 void write_message()
4 {
5     std::ofstream output{"out.txt"};
6     if (!output) {
7         return;
8     }
9     output << "hello\n";
10 }
```

`std::ofstream` 对象离开作用域时会关闭文件。你不需要在每个分支手写关闭逻辑。RAII 的关键不是某个类，而是一个原则：资源的生命周期绑定到对象生命周期。

核心理想

如果一个资源需要释放，就应该有一个对象负责释放。不要让调用者记住“用完后一定要调用 `cleanup`”。这种规则靠人记，迟早会漏。

6.3 `unique_ptr` 表达独占所有权

动态对象不是不能用，而是要表达谁拥有它。独占所有权用 `std::unique_ptr`。

Listing 6.3 独占智能指针表达唯一拥有者

```

1 #include <memory>
2 #include <string>
3
4 struct User {
5     std::string name;
6 };
7
8 std::unique_ptr<User> make_user(std::string name)
9 {
10     return std::make_unique<User>(User{.name = std::move(name)});
11 }

```

`unique_ptr` 不能复制，只能移动。这个限制非常有价值：它让所有权转移在代码里可见。

Listing 6.4 移动表示所有权转移

```

1 std::unique_ptr<User> first = make_user("Ada");
2 std::unique_ptr<User> second = std::move(first);
3 // first 现在不再拥有对象

```

如果一个函数只观察用户，不要接收 `unique_ptr`，而应该接收引用或指针：

Listing 6.5 观察对象不要夺取所有权

```

1 void print_user(const User& user);
2 void maybe_print_user(const User* user);

```

6.4 `shared_ptr` 不是默认选择

`std::shared_ptr` 表达共享所有权。共享所有权意味着对象最后由谁释放取决于所有引用计数的生命周期。这在图结构、缓存、异步回调中 useful，但也更难推理。

常见误区

不要因为“不知道谁拥有”就默认用 `shared_ptr`。这通常说明设计边界还没想清楚。优先用值、引用、`unique_ptr`；确实有共享生命周期时再用 `shared_ptr`。

6.5 移动语义解决的不是“更快复制”

移动语义经常被误解成一种性能魔法。它真正表达的是：一个对象内部拥有的资源可以转交给另一个对象，原对象进入仍然有效但内容不再依赖的状态。以字符串为例：

Listing 6.6 移动字符串

```

1 std::string source = "large payload";
2 std::string target = std::move(source);

```

移动后 `source` 仍然可以析构、赋新值、调用不要求具体内容的成员函数；但不要继续假设它还保存 `"large payload"`。移动不是复制，移动后的对象也不是被销毁的对象。

设计构造函数时，一个常用模式是“按值接收，然后移动进成员”。调用者传右值时可以移动；传左值时会复制一份，语义也清楚。

Listing 6.7 按值接收并移动进成员

```

1 class UserProfile {
2 public:
3     explicit UserProfile(std::string name)
4         : name_{std::move(name)}
5     {
6     }
7
8 private:
9     std::string name_;
10 };

```

这个模式适合构造函数或明确要保存副本的 API。不适合只观察参数的函数。只观察时仍然应该用 `std::string_view` 或 `const std::string&`。

6.6 悬空引用

生命周期错误里最危险的是悬空引用：引用或指针指向的对象已经销毁。

Listing 6.8 返回局部变量引用是错误

```

1 const std::string& bad_name()
2 {
3     std::string name = "Ada";
4     return name; // 错误：函数返回后 name 已销毁
5 }

```

正确做法通常是返回值。现代编译器会做返回值优化或移动，没必要为了“省一次复制”返回悬空引用。

Listing 6.9 返回值拥有自己的生命周期

```

1 std::string make_name()
2 {
3     return "Ada";
4 }

```

6.7 本章边界检查

看到资源管理代码时，问四个问题：

- 谁拥有资源？
- 资源什么时候释放？
- 出错或提前返回时是否释放？
- 接口是在观察资源，还是转移所有权？

这四个问题比背智能指针 API 更重要。

6.8 异常路径下 RAII 为什么成立

手动释放资源最怕“中间退出”。下面这个版本只要第二步失败，就必须记得关闭文件：

Listing 6.10 多出口函数里的手动清理压力

```

1 void write_two_lines_c_style()
2 {

```

```

3 FILE* file = std::fopen("out.txt", "w");
4 if (file == nullptr) {
5     return;
6 }
7 if (std::fprintf(file, "first\n") < 0) {
8     std::fclose(file);
9     return;
10 }
11 if (std::fprintf(file, "second\n") < 0) {
12     std::fclose(file);
13     return;
14 }
15 std::fclose(file);
16 }

```

这个例子里每个失败分支都要重复 `fclose`。分支越多，遗漏概率越高。RAII 把重复规则变成对象析构规则：

Listing 6.11 RAII 让退出路径统一

```

1 void write_two_lines()
2 {
3     std::ofstream output{"out.txt"};
4     if (!output) {
5         return;
6     }
7     output << "first\n";
8     if (!output) {
9         return;
10    }
11    output << "second\n";
12 }

```

无论函数从哪个 `return` 离开，`output` 都会析构。异常路径也一样：栈展开时已经构造成功的局部对象会按逆序析构。这个性质是现代 C++ 资源管理的地基。

6.9 所有权转移要在接口上可见

观察下面两个函数签名：

Listing 6.12 签名暴露所有权语义

```

1 void inspect_user(const User& user);
2 void store_user(std::unique_ptr<User> user);

```

`inspect_user` 告诉调用者：函数只观察对象，调用结束后对象仍归调用者所有。`store_user` 告诉调用者：函数会接管对象所有权，调用后原来的 `unique_ptr` 不再拥有对象。调用点必须写出 `std::move`：

Listing 6.13 调用点显式转移所有权

```

1 auto user = make_user("Ada");
2 inspect_user(*user);
3 store_user(std::move(user));

```


这个 `std::move` 不是把内存“立刻搬走”的命令，它是一个把左值转换成可移动表达式的标记。真正的移动由目标类型的移动构造或移动赋值完成。写 `std::move` 的价值是让读代码的人看见所有权边界。

6.10 移动后的对象该怎么用

移动后的对象仍然有效，但值通常不再值得假设。最安全的后续操作是销毁、重新赋值或调用明确允许的查询：

Listing 6.14 移动后重新赋值再使用

```
1 std::string name = "Ada";
2 std::string stored = std::move(name);
3
4 name = "Grace";
5 std::cout << name << '\n';
```

不要写依赖移动后内容的代码：

Listing 6.15 不要依赖移动后的旧内容

```
1 std::string name = "Ada";
2 std::string stored = std::move(name);
3 // 不要假设 name.empty() 一定为 true
```

标准只保证 `name` 处在可析构、可赋值的有效状态，不保证它具体等于什么。这个规则在容器、字符串、智能指针里都要记住。

6.11 `shared_ptr` 的典型陷阱：循环引用

共享所有权最大的陷阱是循环引用：

Listing 6.16 共享指针循环引用会阻止释放

```
1 struct Node {
2     std::shared_ptr<Node> next;
3     std::shared_ptr<Node> prev;
4 };
5
6 auto first = std::make_shared<Node>();
7 auto second = std::make_shared<Node>();
8 first->next = second;
9 second->prev = first;
```

即使外部的 `first` 和 `second` 离开作用域，两个节点仍然互相持有对方，引用计数不能归零。反向边如果只是观察关系，应该用 `std::weak_ptr`：

Listing 6.17 弱指针表达非拥有反向引用

```
1 struct Node {
2     std::shared_ptr<Node> next;
3     std::weak_ptr<Node> prev;
4 };
```

这再次说明：智能指针不是“高级指针替代品”，而是所有权词汇。选错词汇，生命周期仍然会错。

习题与作业

写一个 `Timer` 类，在构造时记录开始时间，在析构时打印耗时。然后把它放进一个有多个 `return` 分支的函数里，观察每条路径都会打印耗时。这个练习能帮助你把 RAII 从“文件关闭”推广到任意成对动作。

第 7 章

容器、迭代器和标准库思维

7.1 问题入口：去重并保持顺序

给定一组用户 ID，要求去掉重复项，但保留第一次出现的顺序。例如：

Listing 7.1 输入输出示例

```
1 input  = {3, 5, 3, 7, 5, 9};
2 output = {3, 5, 7, 9};
```

朴素写法是每次检查结果数组里是否已有该元素：

Listing 7.2 朴素去重

```
1 std::vector<int> unique_ids;
2 for (int id : ids) {
3     bool seen = false;
4     for (int existing : unique_ids) {
5         if (existing == id) {
6             seen = true;
7             break;
8         }
9     }
10    if (!seen) {
11        unique_ids.push_back(id);
12    }
13 }
```

这段代码正确，但复杂度是平方级。输入很大时，会反复扫描历史结果。

7.2 组合容器表达需求

需求有两个部分：判断是否出现过，保留顺序。哈希集合适合判断出现，向量适合保留顺序。

Listing 7.3 哈希集合加顺序数组

```
1 #include <unordered_set>
2 #include <vector>
3
4 std::vector<int> deduplicate_keep_order(std::span<const int> ids)
5 {
6     std::unordered_set<int> seen;
7     std::vector<int> result;
8     result.reserve(ids.size());
9
10    for (int id : ids) {
11        if (seen.insert(id).second) {
12            result.push_back(id);
13        }
14    }
```

```

15     return result;
16 }

```

`seen.insert(id).second` 表示这次是否真的插入了新元素。这个写法比先 `contains` 再 `insert` 少一次查找。

7.3 vector 不是数组替身

`std::vector` 是连续存储的动态数组。连续存储带来两个结果：遍历快，尾部追加通常快；但中间插入和删除需要移动后面的元素。

Listing 7.4 删除元素时使用 erase-remove

```

1 std::vector<int> values{1, 2, 3, 2, 4};
2 const auto new_end = std::remove(values.begin(), values.end(), 2);
3 values.erase(new_end, values.end());

```

`std::remove` 不会真的缩短容器，它只是把保留元素移动到前面并返回新的逻辑结尾。真正删除尾部残余的是 `erase`。

7.4 迭代器失效

容器操作可能让迭代器、引用和指针失效。比如 `vector` 扩容后，原来指向元素的指针可能全都失效。

Listing 7.5 尾部追加可能让指针失效

```

1 std::vector<int> values;
2 values.push_back(1);
3 int* first = &values[0];
4 values.push_back(2); // 可能扩容
5 // first 可能已经悬空

```

如果提前知道元素数量，用 `reserve` 可以减少扩容次数，但仍然不能把长期稳定地址建立在 `vector` 元素指针上。需要稳定节点地址时，考虑 `std::list`、`std::deque` 或自己管理对象存储，但要先确认真实需求。

7.5 选择容器的经验表

需求	常用容器	注意点
顺序存储、频繁遍历	<code>std::vector</code>	扩容会移动元素
两端插入删除	<code>std::deque</code>	不保证整体连续
按 key 查找	<code>std::unordered_map</code>	平均快，依赖哈希质量
有序遍历	<code>std::map</code>	节点结构，常数较大
唯一集合	<code>std::unordered_set</code>	自定义类型需要 hash

核心思想

标准库容器不是语法点，而是复杂度合同。选择容器前先说清楚：是否需要顺序、是否需要唯一、是否需要稳定地址、主要操作是什么。

7.6 从输入规模推导复杂度

朴素去重为什么慢？用一个具体输入推导。假设所有 ID 都不重复，且一共有 n 个。第一个元素比较 0 次，第二个比较 1 次，第三个比较 2 次，最后一个比较 $n - 1$ 次。总比较次数是：

$$0 + 1 + 2 + \cdots + (n - 1) = \frac{n(n - 1)}{2}$$

这就是平方级。输入 100 个时还不明显，输入 100000 个时就会非常慢。哈希集合版本把“是否见过”变成平均常数时间查询，于是整体接近线性。算法优化不是凭感觉说“哈希快”，而是看历史信息是否被反复扫描。

7.7 reserve 的作用和边界

在去重函数里写 `result.reserve(ids.size())`，不是为了改变复杂度，而是减少扩容。因为最多保留 `ids.size()` 个元素，提前预留这个容量合理。注意 `reserve` 只改变容量，不改变元素个数：

Listing 7.6 `reserve` 不会创建元素

```
1 std::vector<int> values;
2 values.reserve(10);
3 assert(values.size() == 0);
4 values.push_back(42);
5 assert(values.size() == 1);
```

如果写 `resize(10)`，就真的创建了十个元素。两者的区别必须分清：容量是能放多少，大小是已经有多少。

7.8 迭代器失效要看容器合同

不同容器的失效规则不同。入门阶段可以先抓住几个高频点：

- `vector` 扩容会让所有元素引用、指针、迭代器失效。
- `vector` 中间删除会让删除点之后的位置失效。
- `list` 移动节点通常不让其他节点迭代器失效。
- `unordered_map` rehash 可能让迭代器失效，但元素引用通常仍指向元素对象。

不要凭容器名字猜。每次把迭代器保存到后面用，都要问：中间有没有可能插入、删除、扩容或 rehash？LRU 缓存选择 `std::list`，正是因为它需要在移动节点时保持迭代器稳定。

7.9 容器选择案例：排行榜

假设要维护游戏排行榜，需求是按玩家 ID 更新分数，并输出前十名。一个容器很难同时满足所有操作。可以先用 `unordered_map<int, int>` 保存玩家到分数的映射；输出前十时再把条目放进 `vector` 排序：

Listing 7.7 排行榜的两阶段容器设计

```
1 using Scores = std::unordered_map<int, int>;
2
3 std::vector<std::pair<int, int>> top_scores(const Scores& scores)
4 {
5     std::vector<std::pair<int, int>> items(scores.begin(), scores.end());
6     std::ranges::sort(items, [] (const auto& lhs, const auto& rhs) {
7         if (lhs.second != rhs.second) {
8             return lhs.second > rhs.second;
9         }
10        return lhs.first < rhs.first;
11    });
12    if (items.size() > 10) {
13        items.resize(10);
```

```
14     }  
15     return items;  
16 }
```

如果更新非常频繁、查询前十也非常频繁，就可能需要额外维护有序结构。但第一版不要过早复杂化。先让主要操作和数据规模决定容器组合。

习题与作业

给 `deduplicate_keep_order` 写三组测试：全重复、全不重复、混合重复。再把 `unordered_set` 去掉，手写朴素版本，对比两者在 **1000**、**10000**、**100000** 个随机 ID 下的运行时间。

第 8 章

错误处理和接口设计

8.1 问题入口：读取配置文件

假设要读取一个配置文件，里面只有一行端口号。可能发生的错误有：文件不存在、内容为空、不是数字、数字超出范围。朴素写法容易把错误吞掉：

Listing 8.1 吞掉错误的读取函数

```
1 int read_port()
2 {
3     std::ifstream input{"server.conf"};
4     int port = 0;
5     input >> port;
6     return port;
7 }
```

如果文件不存在，函数返回 0。但 0 是错误还是合法端口？调用者不知道。

8.2 错误要进入接口

先定义错误类型：

Listing 8.2 配置读取错误

```
1 enum class ConfigError {
2     file_not_found,
3     empty_file,
4     invalid_number,
5     out_of_range,
6 };
7
8 struct PortConfig {
9     int port = 0;
10 };
11
12 struct PortConfigResult {
13     std::optional<PortConfig> config;
14     std::optional<ConfigError> error;
15 };
```

然后实现读取逻辑：

Listing 8.3 显式返回错误

```
1 #include <fstream>
2 #include <optional>
3 #include <string>
4
5 constexpr int PORT_MIN = 1;
6 constexpr int PORT_MAX = 65535;
```

```

7
8 PortConfigResult read_port_config(const std::string& path)
9 {
10     std::ifstream input{path};
11     if (!input) {
12         return {error = ConfigError::file_not_found};
13     }
14
15     std::string text;
16     if (!(input >> text)) {
17         return {error = ConfigError::empty_file};
18     }
19
20     const std::optional<int> parsed = parse_int(text);
21     if (!parsed.has_value()) {
22         return {error = ConfigError::invalid_number};
23     }
24     if (*parsed < PORT_MIN || *parsed > PORT_MAX) {
25         return {error = ConfigError::out_of_range};
26     }
27     return {.config = PortConfig{.port = *parsed}};
28 }

```

这里没有说异常不好。异常适合无法就地处理、需要跨层传播的错误；显式结果适合调用者马上需要分支处理的错误。关键是不要让错误伪装成普通值。

8.3 接口越小，错误越少

接口设计的第一原则是少暴露。比如一个缓存类不应该同时负责读取文件、解析 JSON、网络刷新、过期策略和日志输出。先定义一个小接口：

Listing 8.4 小接口降低耦合

```

1 class PortProvider {
2 public:
3     explicit PortProvider(PortConfig config)
4         : config_{config}
5     {
6     }
7
8     [[nodiscard]] int port() const noexcept
9     {
10         return this->config_.port;
11     }
12
13 private:
14     PortConfig config_;
15 };

```

它只表达“我能提供端口”。至于端口来自文件、命令行还是测试固定值，不应该被这个类知道。

8.4 异常边界

异常不是洪水猛兽，但要有边界。底层函数可以抛出 `std::runtime_error`，但程序入口应该捕获并转换成可理解的错误消息。

Listing 8.5 程序入口处建立异常边界

```

1 int main()
2 {
3     try {
4         return run_application();
5     } catch (const std::exception& error) {
6         std::cerr << "fatal: " << error.what() << '\n';
7         return 1;
8     }
9 }

```

不要在每一层都捕获再抛出同样的异常。捕获必须有目的：恢复、补充上下文、转换错误类型或终止程序。

常见误区

最差的错误处理是“打印一下然后继续”。如果状态已经不可信，继续执行可能制造更隐蔽的数据损坏。

8.5 从端口号反例推导错误类型

为什么 `read_port` 返回 `0` 不够？因为我们至少有四种不同处理方式。文件不存在时，用户可能需要检查路径；空文件时，可能需要补配置；不是数字时，要指出格式错误；超出范围时，要告诉合法范围。一个 `0` 抹掉了这些差别。

因此错误类型不是为了形式完整，而是为了保留决策所需的信息。还可以给错误转换成人类可读消息：

Listing 8.6 错误枚举转消息

```

1 std::string_view describe(ConfigError error)
2 {
3     switch (error) {
4     case ConfigError::file_not_found:
5         return "config file not found";
6     case ConfigError::empty_file:
7         return "config file is empty";
8     case ConfigError::invalid_number:
9         return "port must be a number";
10    case ConfigError::out_of_range:
11        return "port must be between 1 and 65535";
12    }
13    return "unknown config error";
14 }

```

这个 `switch` 应该靠编译警告帮助维护。如果以后新增错误枚举，却忘了补消息，警告会提醒你。

8.6 错误处理分层

同一个错误在不同层有不同表达。底层读取函数返回结构化错误；命令行入口把错误转换成文本和退出码；测试直接断言错误枚举：

Listing 8.7 入口层转换错误

```

1 int run_server(std::string_view config_path)

```

```

2 {
3     const PortConfigResult result = read_port_config(std::string{config_path});
4     if (result.error.has_value()) {
5         std::cerr << describe(*result.error) << '\n';
6         return 2;
7     }
8     return start_server(result.config->port);
9 }

```

不要让底层函数直接 `std::cerr`。一旦底层打印，图形界面、服务进程和测试环境都被迫接受同一种输出方式。接口返回错误，调用者才有选择。

8.7 optional 适合什么，不适合什么

`std::optional<T>` 适合表达“可能没有值，但没有更多错误信息”。例如按 ID 查找用户：

Listing 8.8 查不到不是异常

```

1 std::optional<UserRecord> find_user(int id);

```

如果失败原因会影响处理，就不要只用 `optional`。读取配置失败显然有不同原因，所以需要错误类型。异常则适合跨多层传播、当前层无法恢复的错误，例如配置语义严重错误、数据库连接失败、程序无法继续启动。

8.8 接口大小与测试成本

比较两个接口：

Listing 8.9 过大的接口难测试

```

1 class ServerManager {
2 public:
3     void load_config();
4     void open_log();
5     void connect_database();
6     void start_threads();
7     void run();
8 };

```

这个类每个测试都可能需要文件、日志、数据库和线程。把端口读取拆成纯函数后，测试只需要临时文件或字符串解析。接口越小，构造测试环境越容易；测试越容易，错误路径越可能被覆盖。

习题与作业

把 `read_port_config` 拆成两个函数：一个从 `std::string_view` 解析端口文本，一个负责读文件。要求文本解析函数不做任何 I/O，并覆盖空字符串、字母、0、65536、合法端口五个测试。

第零部分

泛型、标准库和可组合设计

第 9 章

模板、concept 和泛型设计

9.1 问题入口：同一段逻辑支持不同数字类型

假设要写一个求平均值函数。只支持 `std::vector<int>` 很窄：

Listing 9.1 只能处理 int vector

```
1 double average(const std::vector<int>& values)
2 {
3     int sum = 0;
4     for (int value : values) {
5         sum += value;
6     }
7     return static_cast<double>(sum) / static_cast<double>(values.size());
8 }
```

如果输入是 `double`、`long long` 或 `std::array<int, 4>`，这段函数就不能复用。模板让我们把“类型差异”变成参数。

9.2 函数模板

Listing 9.2 用 span 写数字平均值

```
1 #include <concepts>
2 #include <numeric>
3 #include <span>
4
5 template <std::floating_point Result = double, typename Number>
6 Result average(std::span<const Number> values)
7 {
8     if (values.empty()) {
9         return Result{};
10    }
11    const Number sum = std::accumulate(values.begin(), values.end(), Number{});
12    return static_cast<Result>(sum) / static_cast<Result>(values.size());
13 }
```

这个模板还有局限：`Number` 应该支持加法和默认构造。C++20 的 `concept` 可以把约束写出来。

9.3 用 concept 写约束

Listing 9.3 定义可累加数字概念

```
1 #include <concepts>
2
3 template <typename T>
4 concept AddableNumber = std::default_initializable<T> &&
5     requires(T lhs, T rhs) {
6         { lhs + rhs } -> std::convertible_to<T>;
7     }
```

```

7         };
8
9     template <std::floating_point Result = double, AddableNumber Number>
10    Result checked_average(std::span<const Number> values)
11    {
12        if (values.empty()) {
13            return Result{};
14        }
15        Number sum{};
16        for (const Number& value : values) {
17            sum = sum + value;
18        }
19        return static_cast<Result>(sum) / static_cast<Result>(values.size());
20    }

```

约束不是为了炫技，而是为了把错误提前、把报错变短、把接口意图写清楚。

9.4 模板不是复制粘贴

模板会在使用点实例化。不同类型会生成不同实例，编译时间和错误信息都可能变重。因此模板代码要更克制：不要把巨大业务逻辑塞进模板；能用普通函数处理的部分，放到普通函数里。

Listing 9.4 模板只负责薄薄的类型适配

```

1 double average_vector(const std::vector<int>& values)
2 {
3     return checked_average(std::span<const int>{values});
4 }

```

真实项目中，可以把模板放在接口层，把复杂实现放进非模板函数，减少编译膨胀。

9.5 模板报错怎么读

模板报错常常很长，因为编译器会把实例化链条全部打印出来。读这种报错不要从第一行硬啃到最后一行，而是按三步：

1. 找到你自己的文件路径和行号。
2. 找到第一个“不满足约束”或“没有匹配函数”的位置。
3. 回到模板接口，看传入类型缺少哪种能力。

例如把 `std::string` 传给只接受数值累加的模板，错误本质不是“编译器讨厌字符串”，而是字符串加法结果、默认值、除法转换等能力不符合平均值函数的合同。concept 的价值就是把这种合同前移，让错误更靠近调用点。

9.6 类模板：固定容量缓冲区

Listing 9.5 简单固定容量缓冲区

```

1 #include <array>
2 #include <optional>
3
4 template <typename T, std::size_t CAPACITY>
5 class RingBuffer {
6 public:
7     [[nodiscard]] bool empty() const noexcept
8     {

```

```

9      return this->size_ == 0;
10  }
11
12  [[nodiscard]] bool full() const noexcept
13  {
14      return this->size_ == CAPACITY;
15  }
16
17  bool push(const T& value)
18  {
19      if (this->full()) {
20          return false;
21      }
22      this->data_[this->tail_] = value;
23      this->tail_ = (this->tail_ + 1) % CAPACITY;
24      ++this->size_;
25      return true;
26  }
27
28  std::optional<T> pop()
29  {
30      if (this->empty()) {
31          return std::nullopt;
32      }
33      T value = this->data_[this->head_];
34      this->head_ = (this->head_ + 1) % CAPACITY;
35      --this->size_;
36      return value;
37  }
38
39 private:
40     std::array<T, CAPACITY> data_{};
41     std::size_t head_ = 0;
42     std::size_t tail_ = 0;
43     std::size_t size_ = 0;
44 };

```

这个例子展示了非类型模板参数 `CAPACITY`。容量成为类型的一部分，`RingBuffer<int, 8>` 和 `RingBuffer<int, 16>` 是不同类型。

核心思想

泛型设计的目标不是让一段代码接受所有类型，而是让它接受满足某组明确能力的类型。
concept 就是在接口上写出这组能力。

9.7 从重复函数推导模板

模板不是一上来就该用。先看重复出现在哪里：

Listing 9.6 重复的平均值函数

```

1 double average_ints(std::span<const int> values);
2 double average_doubles(std::span<const double> values);
3 double average_longs(std::span<const long long> values);

```

这三个函数的控制流程相同，只有元素类型不同。于是可以把元素类型提成模板参数。反过来，如果两个函数只是名字相似，但业务规则、错误处理、返回含义都不同，就不应该硬做模板。

9.8 concept 从失败调用里长出来

如果没有约束，下面的调用可能触发很长的模板报错：

Listing 9.7 错误类型不满足平均值语义

```
1 std::vector<std::string> names{"Ada", "Grace"};
2 const double result = checked_average(std::span<const std::string>{names});
```

字符串可以默认构造，也能相加，但不能自然地除以元素个数并转换成浮点平均值。这个反例说明 `AddableNumber` 还不够严格。可以把“可转换成结果类型”也写进约束：

Listing 9.8 更贴近平均值语义的约束

```
1 template <typename T, typename Result>
2 concept AveragableAs = std::default_initializable<T> &&
3     requires(T lhs, T rhs, std::size_t count) {
4         { lhs + rhs } -> std::convertible_to<T>;
5         { static_cast<Result>(lhs) / static_cast<Result>(count) }
6         -> std::convertible_to<Result>;
7     };
```

约束来自真实失败样本，而不是为了把概念写得复杂。每加一个约束，都应该能说出它挡住了哪类误用。

9.9 类模板里的容量边界

`RingBuffer<T, CAPACITY>` 还有一个遗漏：`CAPACITY` 为 0 时，取模会出问题。非类型模板参数也需要约束：

Listing 9.9 用 `requires` 排除零容量

```
1 template <typename T, std::size_t CAPACITY>
2     requires(CAPACITY > 0)
3 class RingBuffer {
4     // ...
5 };
```

这个约束把错误提前到编译期。如果用户写 `RingBuffer<int, 0>`，问题不是运行到“对零取模”才暴露，而是在类型实例化时就被拒绝。模板的一个优势就是把一部分不变量放到类型层。

9.10 模板实现如何避免膨胀

假设一个模板函数里有大段日志、文件处理和复杂业务逻辑。每个类型实例都会生成一份代码，编译也更慢。常见做法是把与类型无关的部分抽到普通函数：

Listing 9.10 模板层只做类型相关转换

```
1 void report_average_result(double value);
2
3 template <typename Number>
4 void report_average(std::span<const Number> values)
5 {
```

```
6     const double value = checked_average(values);
7     report_average_result(value);
8 }
```

这样模板只负责把不同数字类型汇总成 `double`，报告格式等无关逻辑只有一份。泛型代码越靠近基础设施，越要注意编译时间、错误信息和二进制体积。

习题与作业

给 `RingBuffer` 补两个测试：容量为 2 时连续 `push` 三次，第三次应失败；连续 `pop` 三次，第三次应返回 `std::nullopt`。再尝试声明 `RingBuffer<int, 0>`，确认编译期约束生效。

第 10 章

标准算法、ranges 和可组合代码

10.1 问题入口：筛选活跃用户

给定一组用户，筛选活跃用户 ID，并按 ID 排序。朴素写法通常是一堆循环：

Listing 10.1 手写循环筛选和排序

```
1 struct User {
2     int id = 0;
3     bool active = false;
4 };
5
6 std::vector<int> active_ids;
7 for (const User& user : users) {
8     if (user.active) {
9         active_ids.push_back(user.id);
10    }
11 }
12 std::sort(active_ids.begin(), active_ids.end());
```

这段代码没错。标准算法不是为了消灭所有循环，而是把常见意图变成可读的词汇。

10.2 算法表达意图

Listing 10.2 标准算法表达筛选

```
1 std::vector<User> active_users;
2 std::copy_if(users.begin(), users.end(), std::back_inserter(active_users),
3             [](const User& user) {
4                 return user.active;
5             });
```

如果最终只要 ID，还需要一次转换。C++20 ranges 可以写成管道式视图：

Listing 10.3 ranges 视图组合

```
1 #include <ranges>
2
3 auto active_id_view =
4     users
5     | std::views::filter([](const User& user) { return user.active; })
6     | std::views::transform([](const User& user) { return user.id; });
7
8 std::vector<int> ids(active_id_view.begin(), active_id_view.end());
9 std::ranges::sort(ids);
```

视图通常是惰性的：它描述计算，不立即生成新容器。最终构造 `ids` 时才真正遍历。

10.3 谓词和投影

`std::ranges::sort` 支持投影，可以直接按字段排序：

Listing 10.4 按字段排序

```
1 std::ranges::sort(users, std::less<>{}, &User::id);
```

这比手写比较器更短，也更不容易写错。比较器仍然有用，尤其当排序规则涉及多个字段。

10.4 避免悬空视图

`ranges` 的危险点是生命周期。视图不拥有底层数据，如果底层容器销毁，视图就悬空。

Listing 10.5 不要返回依赖局部容器的视图

```
1 auto bad_view()
2 {
3     std::vector<User> users = load_users();
4     return users | std::views::filter([](const User& user) {
5         return user.active;
6     });
7 }
```

正确做法是返回拥有结果的容器，或者让调用者传入数据并保证生命周期。

10.5 什么时候保留普通循环

普通循环仍然重要。涉及复杂状态机、多个输出、短路返回、详细错误上下文时，强行使用算法反而会降低可读性。比如解析协议时，一个明确的 `for` 或 `while` 循环通常比嵌套算法清楚。

核心思想

标准算法和 `ranges` 的价值是表达常见意图：查找、计数、转换、筛选、排序、归约。不是所有循环都要替换，但每个简单循环都值得问一句：这里是不是已有标准词汇？

10.6 从循环变量推导算法词汇

判断一个循环能不能换成标准算法，可以先问它在维护什么状态。下面的循环只维护一个布尔结果：

Listing 10.6 手写检查是否存在活跃用户

```
1 bool has_active = false;
2 for (const User& user : users) {
3     if (user.active) {
4         has_active = true;
5         break;
6     }
7 }
```

这正是 `std::ranges::any_of` 的语义：

Listing 10.7 标准算法表达是否存在

```
1 const bool has_active = std::ranges::any_of(users, [](const User& user) {
2     return user.active;
3 });
```

如果循环维护的是计数，用 `count_if`；维护的是查找位置，用 `find_if`；维护的是转换结果，用 `transform` 或视图。算法不是为了显得高级，而是让读者少读循环细节，直接看到意图。

10.7 视图是借来的计算

`ranges` 视图通常不拥有数据。可以把它理解成一张“计算计划单”：等你遍历它时，它才从原容器取元素、过滤、转换。这个特性节省中间容器，但也带来生命周期要求。

Listing 10.8 视图会反映底层容器变化

```
1 std::vector<User> users{{1, true}, {2, false}};
2 auto active = users | std::views::filter([](const User& user) {
3     return user.active;
4 });
5
6 users.push_back(User{3, true});
7 // 后续遍历 active 时，可能看到新加入的用户
```

这段代码是否安全，还要看 `push_back` 是否导致底层迭代器失效、视图对象如何保存范围。入门阶段先保守：视图不要跨越会修改底层容器结构的代码段长期保存。需要稳定结果时，立刻收集到 `vector`。

10.8 排序规则必须是严格弱序

比较器不是随便返回布尔值。排序要求比较关系稳定、可传递。反例：

Listing 10.9 错误比较器破坏排序语义

```
1 std::ranges::sort(users, [](const User& lhs, const User& rhs) {
2     return lhs.id <= rhs.id; // 错误：相等时也返回 true
3 });
```

当 `lhs.id == rhs.id` 时，`lhs < rhs` 和 `rhs < lhs` 都不应该成立。正确写法是：

Listing 10.10 相等时返回 false

```
1 std::ranges::sort(users, [](const User& lhs, const User& rhs) {
2     return lhs.id < rhs.id;
3 });
```

多字段排序也要按顺序推导：先比较主字段，主字段相等时再比较次字段。

10.9 完整案例：活跃用户报表

把本章内容合成一个小函数：输入用户列表，输出活跃用户 ID，按 ID 升序，不修改原列表。

Listing 10.11 活跃用户 ID 报表

```
1 std::vector<int> active_user_ids(std::span<const User> users)
2 {
3     auto ids_view = users
4         | std::views::filter([](const User& user) {
5             return user.active;
6         })
7         | std::views::transform([](const User& user) {
```

```
8         return user.id;
9     });
10
11     std::vector<int> ids(ids_view.begin(), ids_view.end());
12     std::ranges::sort(ids);
13     ids.erase(std::ranges::unique(ids).begin(), ids.end());
14     return ids;
15 }
```

这里先用视图表达筛选和转换，再收集成拥有结果的 `vector`，最后排序去重。返回值拥有自己的生命周期，不依赖传入的 `users`。

习题与作业

给 `active_user_ids` 加测试：空列表、没有活跃用户、重复 ID、已排序输入、逆序输入。然后用普通循环重写一版，对比哪一版更容易在代码里看出“筛选、转换、排序、去重”四个步骤。

第零部分

工程化：构建、测试、调试和性能

第 11 章

构建、测试和调试

11.1 问题入口：一份源文件走不远

单文件程序适合学习语法，但真实项目很快需要多个源文件、测试和构建脚本。假设项目有一个统计库：

Listing 11.1 最小项目结构

```
1 score_tool/  
2   CMakeLists.txt  
3   include/score/summary.hpp  
4   src/summary.cpp  
5   tests/summary_tests.cpp
```

目录结构表达所有权：公开头文件在 `include`，实现放 `src`，测试放 `tests`。不要把所有文件堆在一个目录里。

11.2 CMake 的基本边界

Listing 11.2 库和测试目标分开

```
1 cmake_minimum_required(VERSION 3.20)  
2 project(score_tool LANGUAGES CXX)  
3  
4 add_library(score_summary src/summary.cpp)  
5 target_compile_features(score_summary PUBLIC cxx_std_20)  
6 target_include_directories(score_summary PUBLIC include)  
7  
8 add_executable(summary_tests tests/summary_tests.cpp)  
9 target_link_libraries(summary_tests PRIVATE score_summary)  
10 enable_testing()  
11 add_test(NAME summary_tests COMMAND summary_tests)
```

目标是 CMake 的核心。不要把编译选项、include 路径和库依赖全局乱撒。谁需要依赖，谁声明依赖。

11.3 测试不是最后补

统计函数天然适合测试：

Listing 11.3 不用测试框架也能先写断言

```
1 #include <cassert>  
2 #include <vector>  
3  
4 int main()  
5 {  
6     const std::vector<int> scores{90, 80, 100};  
7     const auto summary = summarize_scores(scores);
```

```

8   assert(summary.has_value());
9   assert(summary->count == 3);
10  assert(summary->sum == 270);
11  assert(summary->min_score == 80);
12  assert(summary->max_score == 100);
13 }

```

真实项目可以使用 Catch2、GoogleTest 或 doctest。框架不是重点，重点是测试要覆盖边界：空输入、单元素、多元素、极值和错误路径。

11.4 调试先缩小问题

调试不等于到处打印。更可靠的流程是：

1. 复现问题，固定输入。
2. 写一个最小失败测试。
3. 确认失败位置：输入解析、核心逻辑、输出格式还是环境。
4. 修正后保留测试，防止回归。

调试器可以观察变量、调用栈和线程状态；日志可以保留运行历史；断言可以检查不变量。三者用途不同。

Listing 11.4 断言用于检查内部不变量

```

1 #include <cassert>
2
3 void push_value(std::vector<int>& values, int value)
4 {
5     const std::size_t old_size = values.size();
6     values.push_back(value);
7     assert(values.size() == old_size + 1);
8 }

```

断言不是用户错误处理。它适合检查“代码内部本应永远成立”的条件。

11.5 本章边界检查

一个能长期维护的 C++ 项目至少要有：

- 清晰目标：库、程序、测试分别是什么。
- 目标级 include 路径和依赖。
- 可重复运行的测试命令。
- Debug 和 Release 两种构建。
- 编译警告作为质量输入。

核心思想

工程化不是等项目变大后才需要。构建边界、测试入口和调试习惯越早建立，后面修改成本越低。

11.6 从一条编译命令演进到 CMake

单文件时可以直接编译：

Listing 11.5 单文件编译

```

1 g++ -std=c++20 -Wall -Wextra -pedantic main.cpp -o score_tool

```

当项目拆成头文件、源文件和测试后，手写命令会越来越长，而且容易漏依赖。CMake 的价值不是“多一个工具”，而是把目标关系记录下来：库由哪些源文件组成，公开哪些头文件路径，测试链接哪个库。

推荐的构建流程如下：

Listing 11.6 独立构建目录

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build
3 ctest --test-dir build --output-on-failure
```

-S 指源码目录，**-B** 指构建目录。不要把生成文件混进源码目录。Debug 构建适合调试，Release 构建适合性能测量。

11.7 头文件和源文件怎么拆

以成绩统计库为例，头文件只放接口和必要类型：

Listing 11.7 summary.hpp 暴露稳定接口

```
1 #pragma once
2
3 #include <optional>
4 #include <span>
5
6 struct ScoreSummary {
7     std::size_t count = 0;
8     int sum = 0;
9     int min_score = 0;
10    int max_score = 0;
11    double average = 0.0;
12 };
13
14 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores);
```

实现文件放算法细节：

Listing 11.8 summary.cpp 放实现

```
1 #include <score/summary.hpp>
2
3 #include <algorithm>
4 #include <numeric>
5
6 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores)
7 {
8     if (scores.empty()) {
9         return std::nullopt;
10    }
11    const auto [min_it, max_it] = std::minmax_element(scores.begin(), scores.end());
12    const int sum = std::accumulate(scores.begin(), scores.end(), 0);
13    return ScoreSummary{scores.size(), sum, *min_it, *max_it,
14                        static_cast<double>(sum) / scores.size()};
15 }
```


调用者只需要包含头文件，不应该依赖实现文件里的私有细节。这个边界越干净，后面替换实现越容易。

11.8 测试要覆盖失败路径

很多测试只测普通输入，结果错误路径一直没人碰。成绩统计至少要有这些测试：

Listing 11.9 最小测试集合

```
1 void test_empty_scores()
2 {
3     const std::vector<int> scores;
4     assert(!summarize_scores(scores).has_value());
5 }
6
7 void test_single_score()
8 {
9     const std::vector<int> scores{90};
10    const auto summary = summarize_scores(scores);
11    assert(summary.has_value());
12    assert(summary->min_score == 90);
13    assert(summary->max_score == 90);
14    assert(summary->average == 90.0);
15 }
```

测试函数命名要说清行为，而不是叫 `test1`、`test2`。失败时，名字就是第一条诊断信息。

11.9 调试工具各自解决什么

编译警告帮你发现可疑代码；单元测试帮你固定行为；调试器帮你观察某次运行；消毒器帮助发现越界、悬空引用和数据竞争。可以给 Debug 构建加上地址消毒器：

Listing 11.10 为测试目标打开地址消毒器

```
1 target_compile_options(summary_tests PRIVATE -fsanitize=address ↵
   ↵ -fno-omit-frame-pointer)
2 target_link_options(summary_tests PRIVATE -fsanitize=address)
```

这不是替代测试，而是让测试失败得更早、更明确。比如越界访问可能平时“看起来能跑”，在消毒器下会直接报出位置。

习题与作业

把成绩统计项目拆成 `include/score/summary.hpp`、`src/summary.cpp`、`tests/summary_tests.cpp`。要求 `ctest` 能运行测试；再故意把空输入检查删掉，确认测试能失败。

第 12 章

性能、内存和测量

12.1 问题入口：字符串拼接为什么慢

性能优化不能靠感觉。先看一个常见例子：拼接很多小字符串。

Listing 12.1 反复拼接可能频繁分配

```
1 std::string build_line(const std::vector<std::string>& parts)
2 {
3     std::string line;
4     for (const std::string& part : parts) {
5         line += part;
6         line += ',';
7     }
8     return line;
9 }
```

这段代码可能多次扩容。每次扩容都可能分配新内存、复制旧内容。优化前先确认瓶颈，如果它在热路径，可以先预估长度并 `reserve`。

Listing 12.2 预留容量减少分配

```
1 std::string build_line_reserved(const std::vector<std::string>& parts)
2 {
3     std::size_t total_size = 0;
4     for (const std::string& part : parts) {
5         total_size += part.size() + 1;
6     }
7
8     std::string line;
9     line.reserve(total_size);
10    for (const std::string& part : parts) {
11        line += part;
12        line += ',';
13    }
14    return line;
15 }
```

12.2 复杂度和常数都要看

$O(n)$ 不等于一定快， $O(n \log n)$ 不等于一定慢。数据规模、内存局部性、分配次数和分支预测都会影响实际时间。比如 `std::vector` 线性查找小数组，可能比 `std::unordered_set` 更快，因为连续内存和低常数优势明显。

12.3 避免不必要复制

性能问题里最常见的是无意复制：

Listing 12.3 range-for 中复制字符串

```

1 for (auto name : names) {
2     use(name);
3 }

```

如果 `use` 只观察，应该写：

Listing 12.4 只读遍历用 `const` 引用

```

1 for (const auto& name : names) {
2     use(name);
3 }

```

函数参数同理。大的字符串、向量、路径、映射和业务对象，不要在只读场景按值传递。

12.4 测量要可重复

一个最小 benchmark 要避免三个错误：测量太短、把 I/O 算进去、编译器把结果优化掉。下面只是示意：

Listing 12.5 用稳定时钟做粗略测量

```

1 #include <chrono>
2 #include <iostream>
3
4 template <typename Func>
5 void measure(std::string_view name, Func func)
6 {
7     const auto start = std::chrono::steady_clock::now();
8     const auto result = func();
9     const auto finish = std::chrono::steady_clock::now();
10    const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
11        finish - start);
12    std::cout << name << ": " << elapsed.count()
13        << " us, result=" << result << '\n';
14 }

```

严肃 benchmark 应使用专门框架，并固定编译选项、输入规模、运行次数和机器状态。本章先建立测量意识。

12.5 优化前后的检查单

优化不是把代码写得更低级。每次优化前后都应该记录：

- 输入规模：小数据、中数据和目标大数据分别是多少。
- 编译模式：Debug 结果通常不能代表性能。
- 指标：延迟、吞吐、内存占用还是分配次数。
- 正确性：优化后是否跑过同一组测试。
- 可维护性：复杂度增加是否值得。

例如给字符串拼接加 `reserve`，复杂度没有改变，但减少了分配次数，代码仍然容易理解。反过来，如果为了省一个小对象复制引入手写内存池，就必须有测量证明它在目标场景中值得。

12.6 内存布局

对象布局会影响缓存。把频繁一起访问的数据放在连续内存里，通常比到处跳指针更友好。

Listing 12.6 连续数组适合批量处理

```

1 struct Point {
2     double x = 0.0;
3     double y = 0.0;
4 };
5
6 double sum_x(std::span<const Point> points)
7 {
8     double total = 0.0;
9     for (const Point& point : points) {
10         total += point.x;
11     }
12     return total;
13 }

```

更高级场景会讨论结构数组和数组结构。本书先记住：性能不仅是算法，也包括数据如何摆放。

常见误区

不要先优化再证明。先写清楚正确性和边界，再用测量找瓶颈。没有测量的优化，经常只是把代码变复杂。

12.7 从分配次数理解 reserve

字符串拼接慢，根本原因常常不是 `+=` 这个符号，而是容量不够时需要重新分配。假设目标字符串最终长度是 **100000**，如果容量从很小开始逐步增长，就会多次申请新内存，并把旧内容搬过去。每搬一次，历史内容又被读写一遍。

`reserve` 的推理链条是：我们能提前算出最终长度；最终长度给了容器容量上界；提前申请一次能减少后续扩容；扩容减少后，总复制量和分配次数下降。这个优化仍然保持同样的接口和同样的复杂度，维护成本很低，所以优先级高。

12.8 局部性案例：vector 与 list

很多人以为 `std::list` 插入删除快，所以应该更快。对大量顺序遍历来说，事实常常相反：

Listing 12.7 顺序求和更适合连续内存

```

1 int sum_vector(const std::vector<int>& values)
2 {
3     int total = 0;
4     for (int value : values) {
5         total += value;
6     }
7     return total;
8 }

```

`vector` 的元素连续，CPU 能更好地预取。链表每个节点可能分散在内存各处，遍历时不断追指针。算法复杂度同样是 $O(n)$ ，但常数和缓存局部性差很多。除非你真的需要稳定节点和频繁中间插入删除，否则不要默认选择链表。

12.9 避免复制要看语义

不是所有按值都是错。构造函数保存一个字符串时，按值接收再移动是合理的；只读函数按值接收大对象就不合理。判断标准是：函数是否需要拥有一份独立副本。

Listing 12.8 有意按值接收用于保存

```

1 class LogMessage {
2 public:
3     explicit LogMessage(std::string text)
4         : text_{std::move(text)}
5     {
6     }
7
8 private:
9     std::string text_;
10 };

```

如果只是打印：

Listing 12.9 只观察时不要复制

```

1 void print_message(std::string_view text)
2 {
3     std::cout << text << '\n';
4 }

```

`std::string_view` 不拥有字符内容，所以不能长期保存到对象里，除非你能证明底层字符串活得更久。性能优化和生命周期安全必须一起看。

12.10 benchmark 的最小可信做法

粗略测量时，至少让函数重复多次，并把结果用掉：

Listing 12.10 重复运行降低偶然波动

```

1 template <typename Func>
2 void measure_many(std::string_view name, Func func)
3 {
4     constexpr int RUNS = 50;
5     std::int64_t checksum = 0;
6     const auto start = std::chrono::steady_clock::now();
7     for (int i = 0; i < RUNS; ++i) {
8         checksum += func();
9     }
10    const auto finish = std::chrono::steady_clock::now();
11    const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
12        finish - start);
13    std::cout << name << ": " << elapsed.count()
14        << " us, checksum=" << checksum << '\n';
15 }

```

`checksum` 的作用是让计算结果参与可观察输出，降低编译器把整个计算优化掉的可能。真正严肃的性能分析还要用 profiler、benchmark 框架和固定环境；但这个最小版本已经比凭感觉强很多。

习题与作业

实现两个版本的字符串拼接：一个不 `reserve`，一个先计算总长度再 `reserve`。用 Release 构建分别测试 100、10000、100000 个片段。记录时间和输出长度，确认优化没有改变结果。

第零部分

高级专题：并发、异步和大型代码组织

第 13 章

并发、线程和同步

13.1 问题入口：两个线程同时计数

并发最容易制造“偶尔错”的程序。看一个简单计数器：

Listing 13.1 数据竞争

```
1 #include <thread>
2
3 int counter = 0;
4
5 void increment_many()
6 {
7     for (int i = 0; i < 100000; ++i) {
8         ++counter;
9     }
10 }
```

如果两个线程同时执行，结果未必是 200000。`++counter` 不是不可分割动作，它通常包含读取、加一、写回。两个线程交错时会丢更新。这是数据竞争，行为不可靠。

13.2 mutex 保护共享状态

Listing 13.2 用 mutex 保护计数器

```
1 #include <mutex>
2
3 class Counter {
4 public:
5     void increment()
6     {
7         std::lock_guard<std::mutex> lock{this->mutex_};
8         ++this->value_;
9     }
10
11     [[nodiscard]] int value() const
12     {
13         std::lock_guard<std::mutex> lock{this->mutex_};
14         return this->value_;
15     }
16
17 private:
18     mutable std::mutex mutex_;
19     int value_ = 0;
20 };
```

`std::lock_guard` 是 RAII：构造时加锁，析构时解锁。即使中途抛异常，也会释放锁。

13.3 atomic 适合简单共享变量

简单计数可以用原子类型：

Listing 13.3 atomic 计数

```

1 #include <atomic>
2
3 std::atomic<int> counter{0};
4
5 void increment_many()
6 {
7     for (int i = 0; i < 100000; ++i) {
8         counter.fetch_add(1, std::memory_order_relaxed);
9     }
10 }
```

`memory_order_relaxed` 只保证计数本身原子，不建立额外同步关系。入门阶段不要随意选择内存序。默认顺序一致最容易推理；只有在测量证明需要时，才进入更细的内存序设计。

13.4 条件变量表达等待

线程间等待不要忙等。生产者消费者可以用 `std::condition_variable`。

Listing 13.4 条件变量等待队列非空

```

1 #include <condition_variable>
2 #include <mutex>
3 #include <queue>
4
5 class WorkQueue {
6 public:
7     void push(int value)
8     {
9         {
10             std::lock_guard<std::mutex> lock{this->mutex_};
11             this->queue_.push(value);
12         }
13         this->ready_.notify_one();
14     }
15
16     int pop()
17     {
18         std::unique_lock<std::mutex> lock{this->mutex_};
19         this->ready_.wait(lock, [this] {
20             return !this->queue_.empty();
21         });
22         const int value = this->queue_.front();
23         this->queue_.pop();
24         return value;
25     }
26
27 private:
28     std::mutex mutex_;
29     std::condition_variable ready_;
```



```

30     std::queue<int> queue_;
31 };

```

`wait` 必须带谓词，因为线程可能被虚假唤醒。谓词是等待条件的正式表达。

13.5 线程退出也是协议

生产者消费者队列如果没有退出协议，消费者可能永远阻塞。可以加入关闭标志：

Listing 13.5 带关闭协议的队列片段

```

1  class ClosableQueue {
2  public:
3      void close()
4      {
5          {
6              std::lock_guard<std::mutex> lock{this->mutex_};
7              this->closed_ = true;
8          }
9          this->ready_.notify_all();
10     }
11
12     std::optional<int> pop()
13     {
14         std::unique_lock<std::mutex> lock{this->mutex_};
15         this->ready_.wait(lock, [this] {
16             return this->closed_ || !this->queue_.empty();
17         });
18         if (this->queue_.empty()) {
19             return std::nullopt;
20         }
21         const int value = this->queue_.front();
22         this->queue_.pop();
23         return value;
24     }
25
26 private:
27     std::mutex mutex_;
28     std::condition_variable ready_;
29     std::queue<int> queue_;
30     bool closed_ = false;
31 };

```

关闭不是简单地杀线程，而是让等待方能观察到“不会再有新任务”的状态，并自然退出循环。

13.6 本章边界检查

写并发代码时至少检查：

- 哪些状态被多个线程共享？
- 每个共享状态由哪个锁或原子保护？
- 是否存在锁顺序反转导致死锁？
- 条件变量是否用谓词等待？
- 线程退出和资源释放路径是否明确？

核心思想

并发编程的第一原则是减少共享。能在线程内拥有的数据，不共享；必须共享的数据，用清晰同步边界保护。

13.7 从一次丢更新推导数据竞争

`++counter` 可以拆成三步：读取旧值、计算新值、写回新值。假设两个线程都看到旧值是 10：

Listing 13.6 一次丢更新的交错过程

```
1 线程 A: read counter -> 10
2 线程 B: read counter -> 10
3 线程 A: write 11
4 线程 B: write 11
```

执行了两次自增，结果却只增加一次。这不是“线程不够快”，而是没有同步保护共享状态。只要两个线程同时访问同一内存位置，至少一个写，并且没有同步，就形成数据竞争。

13.8 锁保护的是不变量，不是一行代码

给计数器加锁很简单，但真实对象往往有多个成员。锁要保护的是这些成员之间的不变量。比如银行账户转账，不能只给扣款和加款分别加锁后随意组合，还要考虑两个账户的锁顺序。

死锁的典型反例：

Listing 13.7 锁顺序反转可能死锁

```
1 void transfer(Account& from, Account& to, int amount)
2 {
3     std::lock_guard<std::mutex> first{from.mutex};
4     std::lock_guard<std::mutex> second{to.mutex};
5     from.balance -= amount;
6     to.balance += amount;
7 }
```

如果线程一执行 `transfer(a, b, 10)`，线程二同时执行 `transfer(b, a, 20)`，两个线程可能各自拿住一个锁，然后等待对方释放。可以用 `std::scoped_lock` 一次锁住多个互斥量：

Listing 13.8 作用域锁同时获取多个锁

```
1 void transfer(Account& from, Account& to, int amount)
2 {
3     std::scoped_lock lock{from.mutex, to.mutex};
4     from.balance -= amount;
5     to.balance += amount;
6 }
```

更完整的设计还要处理 `from` 和 `to` 是同一个账户、余额不足、异常安全等业务规则。并发代码不能脱离不变量讨论。

13.9 jthread 让线程生命周期更安全

C++20 提供 `std::jthread`，析构时会请求停止并自动 join。相比裸 `std::thread`，它更符合 RAII：

Listing 13.9 jthread 自动 join

```

1 #include <chrono>
2 #include <thread>
3
4 void worker(std::stop_token token)
5 {
6     while (!token.stop_requested()) {
7         do_one_unit_of_work();
8     }
9 }
10
11 void run_background()
12 {
13     std::jthread thread{worker};
14     std::this_thread::sleep_for(std::chrono::seconds{1});
15 } // thread 析构时请求停止并 join

```

这不等于可以随便启动后台线程。停止请求只是协作信号，工作函数必须定期检查它。如果线程阻塞在条件变量或 I/O 上，还需要设计唤醒和关闭协议。

13.10 条件变量为什么要谓词

条件变量可能虚假唤醒，也可能在通知前条件已经变化。谓词把“醒来后是否真的能继续”写成可重复检查的条件：

Listing 13.10 wait 等价于循环检查

```

1 while (queue.empty() && !closed) {
2     ready.wait(lock);
3 }

```

带谓词的 `wait` 帮你写出这个循环。不要用 `if` 替代 `while`，因为醒来不代表条件一定满足。

习题与作业

给 `ClosableQueue` 增加一个 `push` 规则：关闭后不允许再插入。设计返回值或异常策略，并测试三个场景：正常 `push/pop`、关闭空队列、关闭后 `push`。

第 14 章

大型代码组织和演进

14.1 问题入口：一个 Manager 类吞掉项目

很多项目会出现一个 **AppManager**：它读配置、连数据库、处理请求、写日志、管理缓存、启动线程。短期看方便，长期看每个修改都要碰它，测试也越来越难。

架构不是画复杂图，而是控制依赖方向。先把系统拆成几个角色：

- **配置**：读取和验证启动参数。
- **领域逻辑**：处理核心业务，不知道文件和网络。
- **适配器**：把文件、数据库、HTTP 等外部世界接进来。
- **入口**：组装对象，建立异常和日志边界。

14.2 依赖接口而不是具体环境

假设业务逻辑需要读取用户信息。不要让它直接依赖文件路径：

Listing 14.1 用接口隔离外部数据源

```
1 #include <optional>
2 #include <string>
3
4 struct UserRecord {
5     int id = 0;
6     std::string name;
7 };
8
9 class UserRepository {
10 public:
11     virtual ~UserRepository() = default;
12     [[nodiscard]] virtual std::optional<UserRecord> find_by_id(int id) const = 0;
13 };
```

业务服务依赖这个接口：

Listing 14.2 业务逻辑依赖抽象

```
1 class GreetingService {
2 public:
3     explicit GreetingService(const UserRepository& repository)
4         : repository_{repository}
5     {
6     }
7
8     [[nodiscard]] std::string greeting_for(int id) const
9     {
10         const std::optional<UserRecord> user = this->repository_.find_by_id(id);
11         if (!user.has_value()) {
12             return "unknown user";
13         }
14         return "hello, " + user->name;
```

```

15     }
16
17 private:
18     const UserRepository& repository_;
19 };

```

测试时可以传入内存实现；生产环境可以传入文件或数据库实现。

14.3 继承要克制

上面的接口用了虚函数，是为了隔离外部数据源。不要把所有变化点都做成继承。很多时候，普通值对象、函数对象、模板参数或构造函数注入更简单。

Listing 14.3 小策略可以用函数对象

```

1 using PriceRule = std::function<int(int base_price)>;
2
3 int final_price(int base_price, const PriceRule& rule)
4 {
5     return rule(base_price);
6 }

```

如果策略在热路径，`std::function` 的类型擦除成本可能需要测量；也可以用模板参数。架构决策要和性能约束一起看。

14.4 模块边界和头文件

头文件是依赖传播的入口。一个头文件包含太多其他头文件，会让编译变慢，也会放大耦合。优先在头文件里暴露必要类型，在源文件里包含实现细节。

Listing 14.4 头文件保持小而稳定

```

1 #pragma once
2
3 #include <optional>
4 #include <string>
5
6 struct UserRecord;
7
8 class UserRepository {
9 public:
10     virtual ~UserRepository() = default;
11     [[nodiscard]] virtual std::optional<UserRecord> find_by_id(int id) const = 0;
12 };

```

不是所有类型都能前置声明，例如按值成员需要完整类型。知道这个边界，才能合理拆分。

14.5 演进优先级

当项目开始变大，优先处理这些问题：

1. 重复业务规则：抽成清晰函数或类型。
2. 隐式资源所有权：改成 RAII 和明确所有者。
3. 巨大函数：按阶段拆分。
4. 巨大类：按角色拆分。
5. 难测试外部依赖：通过接口或适配器隔离。

核心理想

高级 C++ 不是把模板、继承和并发全用上，而是在复杂系统里让依赖方向、资源生命周期和错误边界仍然清楚。

14.6 从难测试代码推导依赖方向

如果 `GreetingService` 自己打开文件、解析数据、拼接问候语，测试一次问候语就要准备文件路径和文件内容。难测试不是测试人员的问题，而是依赖方向暴露了设计问题：业务逻辑依赖了外部环境。

把依赖倒过来后，业务逻辑只依赖 `UserRepository` 抽象。测试可以写一个内存仓库：

Listing 14.5 测试用内存仓库

```
1 class InMemoryUserRepository final : public UserRepository {
2 public:
3     void add(UserRecord user)
4     {
5         this->users_[user.id] = std::move(user);
6     }
7
8     [[nodiscard]] std::optional<UserRecord> find_by_id(int id) const override
9     {
10         const auto found = this->users_.find(id);
11         if (found == this->users_.end()) {
12             return std::nullopt;
13         }
14         return found->second;
15     }
16
17 private:
18     std::unordered_map<int, UserRecord> users_;
19 };
```

这段测试实现不需要文件和数据库。它也逼着接口保持小：如果 `UserRepository` 暴露十几个无关方法，测试替身就会很重。

14.7 入口负责组装，不负责业务

程序入口可以理解成装配层：

Listing 14.6 入口组装真实依赖

```
1 int run_application(const Options& options)
2 {
3     FileUserRepository repository{options.user_file};
4     GreetingService service{repository};
5     std::cout << service.greeting_for(options.user_id) << '\n';
6     return 0;
7 }
```

这里可以出现文件路径、命令行选项、日志和异常边界，因为入口本来就在系统边缘。业务服务内部则不应该知道这些环境细节。这个边界能让核心代码在没有真实环境的情况下测试。

14.8 什么时候不用虚函数

虚函数适合运行期替换实现，例如生产环境和测试环境用不同仓库。如果策略在编译期就确定，模板或函数对象更轻：

Listing 14.7 编译期策略组合

```
1 template <typename DiscountRule>
2 int final_price(int base_price, DiscountRule rule)
3 {
4     return rule(base_price);
5 }
6
7 const int price = final_price(1000, [](int base) {
8     return base - base / 10;
9 });
```

选择虚函数、模板还是 `std::function`，要看变化发生在运行期还是编译期、是否需要类型擦除、是否在热路径、报错和编译时间是否可接受。高级设计不是某个模式永远正确，而是能说出取舍。

14.9 演进时不要大爆炸重写

大型代码最怕“一次性重写”。更稳的演进方式是先找最痛的边界，建立薄接口，然后逐步迁移调用者。例如一个巨大 `AppManager` 可以先抽出配置读取：

Listing 14.8 先抽出一个窄能力

```
1 class ConfigLoader {
2 public:
3     [[nodiscard]] Config load(const std::string& path) const;
4 };
```

下一步再抽日志、缓存、业务服务。每抽一步都要保留测试，让行为不漂移。架构改进不是为了目录好看，而是为了让下一次修改更小、更可验证。

习题与作业

选一个你写过的单文件程序，画出三类代码：输入输出、核心计算、格式化展示。先只抽出核心计算函数并写测试，不要急着引入类。等函数边界稳定后，再判断是否需要对象封装。

第零部分

综合项目：把语言能力落到工程

第 15 章

项目一：命令行词频统计器

15.1 项目目标

这一章把前面学过的内容放进一个小工具：读取文本文件，统计单词出现次数，输出出现次数最高的前 *k* 个单词。这个项目覆盖文件 RAII、字符串处理、哈希表、排序、错误处理和命令行参数。目标命令如下：

Listing 15.1 词频工具目标用法

```
1 wordfreq --input article.txt --top 20
```

输出示例：

Listing 15.2 词频输出示例

```
1 the 128
2 system 42
3 memory 31
```

15.2 第一版：把文件读进字符串

最简单的做法是逐词读取：

Listing 15.3 逐词读取文件

```
1 #include <fstream>
2 #include <string>
3 #include <vector>
4
5 std::vector<std::string> read_words(const std::string& path)
6 {
7     std::ifstream input{path};
8     std::vector<std::string> words;
9     std::string word;
10    while (input >> word) {
11        words.push_back(word);
12    }
13    return words;
14 }
```

这段代码利用 `std::ifstream` 的 RAII 自动关闭文件。但它还不够：文件打不开时返回空数组，调用者无法区分“空文件”和“文件不存在”；标点也没有处理，“`system`”和“`system,`”会被当成两个词。

15.3 定义错误和清洗规则

先定义读取结果：

Listing 15.4 读取结果类型

```

1 struct WordReadError {
2     std::string message;
3 };
4
5 struct WordReadResult {
6     std::vector<std::string> words;
7     std::optional<WordReadError> error;
8 };

```

单词清洗规则先保持简单：保留字母和数字，统一转小写，其他字符当作分隔。

Listing 15.5 清洗一行文本

```

1 #include <cctype>
2
3 std::vector<std::string> split_words(std::string_view line)
4 {
5     std::vector<std::string> words;
6     std::string current;
7     for (unsigned char ch : line) {
8         if (std::isalnum(ch)) {
9             current.push_back(static_cast<char>(std::tolower(ch)));
10            continue;
11        }
12        if (!current.empty()) {
13            words.push_back(current);
14            current.clear();
15        }
16    }
17    if (!current.empty()) {
18        words.push_back(current);
19    }
20    return words;
21 }

```

这里用 `unsigned char` 是细节。很多 `cctype` 函数要求参数能表示为 `unsigned char` 或 EOF，直接传负的 `char` 可能出问题。

15.4 统计和排序

统计用 `unordered_map`，排序用 `vector`。

Listing 15.6 统计词频

```

1 #include <unordered_map>
2
3 using WordCounts = std::unordered_map<std::string, int>;
4
5 WordCounts count_words(std::span<const std::string> words)
6 {
7     WordCounts counts;
8     for (const std::string& word : words) {
9         ++counts[word];
10    }

```

```

11     return counts;
12 }

```

为了排序，把哈希表转成数组：

Listing 15.7 取 top k 高频词

```

1  #include <algorithm>
2
3  struct WordCount {
4      std::string word;
5      int count = 0;
6  };
7
8  std::vector<WordCount> top_words(const WordCounts& counts, std::size_t limit)
9  {
10     std::vector<WordCount> items;
11     items.reserve(counts.size());
12     for (const auto& [word, count] : counts) {
13         items.push_back(WordCount{.word = word, .count = count});
14     }
15
16     std::ranges::sort(items, [])(const WordCount& lhs, const WordCount& rhs) {
17         if (lhs.count != rhs.count) {
18             return lhs.count > rhs.count;
19         }
20         return lhs.word < rhs.word;
21     });
22
23     if (items.size() > limit) {
24         items.resize(limit);
25     }
26     return items;
27 }

```

排序规则必须完整：次数高的排前面；次数相同按字典序。这让输出稳定，测试也稳定。

15.5 复杂度和替代方案

设单词总数为 n ，不同单词数为 m 。统计平均复杂度是 $O(n)$ ，排序是 $O(m \log m)$ 。如果 k 很小而 m 很大，可以用小顶堆把排序降到 $O(m \log k)$ 。但第一版项目更重视正确性和可读性，全排序完全可以接受。

15.6 测试清单

- 空文件：输出为空。
- 大小写：C++ 和 c++ 合并。
- 标点：memory, memory. 计为同一个词。
- 次数相同：按字典序输出。
- 文件不存在：返回明确错误。

核心思想

一个小工具也要有结构：输入和清洗、统计、排序、输出分开。这样每一步都能单独测试，后续替换算法也不会牵动整段程序。

15.7 把需求拆成可测试流水线

词频工具看起来是一个命令，其实可以拆成五步：

1. 解析命令行，得到输入路径和 `top` 数量。
2. 读取文件，每次拿到一行文本。
3. 把一行文本清洗成单词。
4. 统计所有单词出现次数。
5. 排序、截断并输出。

拆分的标准是：每一步都有清楚输入输出，可以单独测试。比如 `split_words("Cpp, CPP!")` 应该得到两个 `cpp`；这个测试不需要真实文件。等清洗规则可靠后，再测试文件读取。

15.8 读取文件时保留错误

前面的 `read_words` 会把文件不存在和空文件都变成空数组。改进版应该显式检查文件打开：

Listing 15.8 逐行读取并返回错误

```
1 WordReadResult read_words_from_file(const std::string& path)
2 {
3     std::ifstream input{path};
4     if (!input) {
5         return {error = WordReadError{"cannot open input file"}};
6     }
7
8     WordReadResult result;
9     std::string line;
10    while (std::getline(input, line)) {
11        std::vector<std::string> words = split_words(line);
12        result.words.insert(result.words.end(),
13                            std::make_move_iterator(words.begin()),
14                            std::make_move_iterator(words.end()));
15    }
16    if (input.bad()) {
17        return {error = WordReadError{"failed while reading input file"}};
18    }
19    return result;
20 }
```

这里用 `std::getline` 是为了按行清洗标点；用 `make_move_iterator` 是因为临时 `words` 里的字符串已经不再需要，可以移动到结果里。即便如此，第一版仍然把所有单词存进内存。若文件很大，下一步可以改成边读边统计。

15.9 流式版本减少内存

更好的结构是读取一行就更新计数，不必保存所有单词：

Listing 15.9 边读边统计

```
1 struct CountResult {
2     WordCounts counts;
```

```

3     std::optional<WordReadError> error;
4 };
5
6 CountResult count_words_in_file(const std::string& path)
7 {
8     std::ifstream input{path};
9     if (!input) {
10         return {.error = WordReadError{"cannot open input file"}};
11     }
12
13     CountResult result;
14     std::string line;
15     while (std::getline(input, line)) {
16         for (std::string& word : split_words(line)) {
17             ++result.counts[word];
18         }
19     }
20     if (input.bad()) {
21         return {.error = WordReadError{"failed while reading input file"}};
22     }
23     return result;
24 }

```

这个版本的峰值内存主要由不同单词数决定，而不是所有单词总数。我们不是为了炫耀优化，而是从数据规模推出结构：如果文本很大，保存全部中间单词没有必要。

15.10 主程序如何连接模块

入口函数把前面步骤串起来：

Listing 15.10 词频工具主流程

```

1 int run_wordfreq(const Options& options)
2 {
3     const CountResult counted = count_words_in_file(options.input_path);
4     if (counted.error.has_value()) {
5         std::cerr << counted.error->message << '\n';
6         return 2;
7     }
8
9     const std::vector<WordCount> top = top_words(counted.counts, options.top);
10    for (const WordCount& item : top) {
11        std::cout << item.word << " " << item.count << '\n';
12    }
13    return 0;
14 }

```

这个函数仍然很短，因为细节已经在可测试函数里。入口处理错误消息和输出，核心函数处理数据规则。以后如果要输出 JSON，只需要替换输出层；如果要支持停用词，只需要在清洗或统计之间加一步过滤。

15.11 项目验收标准

一个可交付的小工具不只是“我本机能跑”。至少要满足：

- 命令行缺少 `-input` 或 `-top` 时给出错误。
- 文件打不开时返回非零退出码。
- 标点、大小写、空行处理符合文档规则。
- 次数相同时输出顺序稳定。
- 核心函数有测试，测试不依赖真实工作目录。

习题与作业

给词频工具增加 `-min-length N` 参数，只统计长度至少为 `N` 的单词。先写清楚 `N = 0`、缺值、非数字、负数分别怎么处理，再改参数解析和统计流程。

第 16 章

项目二：实现一个 LRU 缓存

16.1 项目目标

LRU 缓存是面试题，也是工程中常见结构。需求是：缓存最多保存 `capacity` 个键值对；每次访问一个键，这个键变成最新；插入新键时如果容量满，删除最久未使用的键。

一个可用接口如下：

Listing 16.1 LRU 缓存接口

```
1 template <typename Key, typename Value>
2 class LruCache {
3 public:
4     explicit LruCache(std::size_t capacity);
5     [[nodiscard]] std::optional<Value> get(const Key& key);
6     void put(Key key, Value value);
7     [[nodiscard]] std::size_t size() const noexcept;
8 };
```

16.2 为什么不能只用 `unordered_map`

`unordered_map` 能快速按键查找，但不知道谁最久没用。只用 `vector` 能维护顺序，但查找是线性的。LRU 需要两个结构组合：

- `std::list` 维护新旧顺序，表头表示最新。
- `std::unordered_map` 从 `key` 定位到链表节点。

16.3 节点和成员

Listing 16.2 LRU 的内部结构

```
1 #include <list>
2 #include <optional>
3 #include <unordered_map>
4
5 template <typename Key, typename Value>
6 class LruCache {
7 private:
8     struct Entry {
9         Key key;
10        Value value;
11    };
12
13    using EntryList = std::list<Entry>;
14    using EntryIterator = typename EntryList::iterator;
15
16 public:
17     explicit LruCache(std::size_t capacity)
18         : capacity_{capacity}
```

```

19     {
20     }
21
22 private:
23     std::size_t capacity_ = 0;
24     EntryList entries_;
25     std::unordered_map<Key, EntryIterator> index_;
26 };

```

链表迭代器在移动节点时保持有效，这是选择 `std::list` 的关键原因。

16.4 访问时移动到表头

Listing 16.3 get 操作

```

1 template <typename Key, typename Value>
2 std::optional<Value> LruCache<Key, Value>::get(const Key& key)
3 {
4     const auto found = this->index_.find(key);
5     if (found == this->index_.end()) {
6         return std::nullopt;
7     }
8
9     this->entries_.splice(this->entries_.begin(), this->entries_, found->second);
10    return found->second->value;
11 }

```

`splice` 不复制节点，只把已有节点移动到新位置。移动后迭代器仍指向同一个节点。

16.5 插入和淘汰

Listing 16.4 put 操作

```

1 template <typename Key, typename Value>
2 void LruCache<Key, Value>::put(Key key, Value value)
3 {
4     if (this->capacity_ == 0) {
5         return;
6     }
7
8     const auto found = this->index_.find(key);
9     if (found != this->index_.end()) {
10        found->second->value = std::move(value);
11        this->entries_.splice(this->entries_.begin(), this->entries_, found->second);
12        return;
13    }
14
15    this->entries_.push_front(Entry{.key = std::move(key), .value = std::move(value)});
16    this->index_[this->entries_.front().key] = this->entries_.begin();
17
18    if (this->entries_.size() > this->capacity_) {
19        const Key evicted_key = this->entries_.back().key;

```



```

20     this->index_.erase(evicted_key);
21     this->entries_.pop_back();
22 }
23 }

```

这个版本为清晰牺牲了一点细节：淘汰时复制了 **Key**。如果 key 很大，可以先移动或用节点引用优化，但要小心 **pop_back** 后引用失效。教材第一版先让所有权和顺序正确。

16.6 不变量

LRU 的核心不变量：

- `entries_.size() == index_.size()`。
- `index_` 中每个迭代器都指向 `entries_` 中的节点。
- 链表表头是最新访问项，表尾是最久未使用项。
- 容量为 0 时，缓存永远为空。

测试必须围绕这些不变量设计。

16.7 测试用例

Listing 16.5 LRU 行为测试思路

```

1 LruCache<int, std::string> cache{2};
2 cache.put(1, "one");
3 cache.put(2, "two");
4 assert(cache.get(1) == "one"); // 1 变成最新
5 cache.put(3, "three");         // 淘汰 2
6 assert(!cache.get(2).has_value());
7 assert(cache.get(1) == "one");
8 assert(cache.get(3) == "three");

```

常见误区

LRU 最容易错在“查找表和链表不同步”。每次插入、访问、淘汰后，都要能解释两者如何保持一致。

16.8 从样例手推状态变化

先不用代码，手推一个容量为 2 的缓存。链表从左到右表示“最新到最旧”。

Listing 16.6 LRU 状态推演

```

1 put(1, one)      [1]
2 put(2, two)      [2, 1]
3 get(1) -> one    [1, 2]
4 put(3, three)    [3, 1] 淘汰 2
5 get(2) -> miss   [3, 1]
6 put(1, uno)      [1, 3] 更新 1 并移动到最新

```

这段推演给出实现规则：新增节点放表头；命中节点移动到表头；容量超过时删除表尾；更新已有键时不能增加大小。任何实现只要违背其中一条，都会在样例里暴露。

16.9 put 的细节拆解

put 有三条路径。第一，容量为零，任何插入都无效。第二，键已存在，只更新值并移动节点。第三，键不存在，插入新节点，必要时淘汰旧节点。把这三条路径分清，代码就不会乱。

为了减少淘汰时复制 **key**，可以先读取表尾 **key**，再擦除映射，最后弹出节点。注意顺序不能反：

Listing 16.7 淘汰顺序必须保证 **key** 仍可读

```
1 const Key& evicted_key = this->entries_.back().key;
2 this->index_.erase(evicted_key);
3 this->entries_.pop_back();
```

这里在 **pop_back** 之前使用引用是安全的；**pop_back** 之后引用立刻失效，所以不能再访问。教材前面的复制版本更保守，适合先学正确性；这个版本展示如何在清楚生命周期的前提下减少复制。

16.10 完整类实现

把接口、成员和操作合在一起：

Listing 16.8 可用的 LRU 缓存实现

```
1 template <typename Key, typename Value>
2 class LruCache {
3 private:
4     struct Entry {
5         Key key;
6         Value value;
7     };
8
9     using EntryList = std::list<Entry>;
10    using EntryIterator = typename EntryList::iterator;
11
12 public:
13     explicit LruCache(std::size_t capacity)
14         : capacity_{capacity}
15     {
16     }
17
18     [[nodiscard]] std::optional<Value> get(const Key& key)
19     {
20         const auto found = this->index_.find(key);
21         if (found == this->index_.end()) {
22             return std::nullopt;
23         }
24         this->entries_.splice(this->entries_.begin(), this->entries_, found->second);
25         return found->second->value;
26     }
27
28     void put(Key key, Value value)
29     {
30         if (this->capacity_ == 0) {
31             return;
32         }
33
34         const auto found = this->index_.find(key);
```

```

35     if (found != this->index_.end()) {
36         found->second->value = std::move(value);
37         this->entries_.splice(this->entries_.begin(), this->entries_, found->
↵ second);
38         return;
39     }
40
41     this->entries_.push_front(Entry{std::move(key), std::move(value)});
42     this->index_[this->entries_.front().key] = this->entries_.begin();
43     if (this->entries_.size() > this->capacity_) {
44         const Key& evicted_key = this->entries_.back().key;
45         this->index_.erase(evicted_key);
46         this->entries_.pop_back();
47     }
48 }
49
50 [[nodiscard]] std::size_t size() const noexcept
51 {
52     return this->entries_.size();
53 }
54
55 private:
56     std::size_t capacity_ = 0;
57     EntryList entries_;
58     std::unordered_map<Key, EntryIterator> index_;
59 };

```

16.11 为什么返回 `optional<Value>` 而不是引用

`get` 返回 `std::optional<Value>` 会复制值。对于大对象，这可能有成本。为什么教材第一版仍然这么写？因为它简单、安全，调用者拿到的是独立值，不会因为后续缓存淘汰变成悬空引用。工程里可以设计两个接口：

Listing 16.9 按需提供引用接口

```

1 [[nodiscard]] Value* find(const Key& key);
2 [[nodiscard]] const Value* find(const Key& key) const;

```

指针返回 `nullptr` 表示未命中，命中时不复制。但文档必须说明：返回指针只在缓存下一次可能修改前有效。接口越高效，生命周期要求通常越严格。

16.12 测试不变量

除了行为测试，还可以在测试版本里暴露一个检查函数，验证映射和链表同步：

Listing 16.10 测试辅助不变量检查

```

1 [[nodiscard]] bool invariant_holds() const
2 {
3     if (this->entries_.size() != this->index_.size()) {
4         return false;
5     }
6     for (auto it = this->entries_.begin(); it != this->entries_.end(); ++it) {
7         const auto found = this->index_.find(it->key);

```

```
8         if (found == this->index_.end() || found->second != it) {
9             return false;
10        }
11    }
12    return true;
13 }
```

生产代码不一定要公开这个函数，但测试思路必须围绕它：每次 `put`、`get`、淘汰和更新后，链表与哈希表都应该描述同一组条目。

16.13 复杂度和替代设计

`get` 和 `put` 平均是 $O(1)$ ，前提是哈希表表现正常，链表移动是常数时间。替代方案包括：

- 只用 `vector`：顺序清楚，但查找和移动都是线性。
- 只用 `unordered_map`：查找快，但不知道谁最旧。
- `unordered_map` 加自定义双向链表：可控性更高，但实现和测试成本更高。

标准库组合已经能满足大多数场景。只有在性能测量证明 `std::list` 节点分配成为瓶颈时，才值得考虑自定义存储。

习题与作业

给 LRU 增加 `contains` 和 `erase`。要求 `contains` 不改变新旧顺序，`erase` 同时删除链表节点和哈希表项。写测试覆盖：删除不存在的 key、删除最新 key、删除最旧 key、删除后再次插入。

附录 A

学习检查清单

A.1 入门阶段

- 能解释编译、链接和运行的区别。
- 能区分对象、值、名字、引用和指针。
- 能写出不越界的 `vector` 遍历。
- 能把读输入、处理数据、输出结果拆成不同函数。
- 能用 `const&` 避免只读场景的复制。

A.2 中级阶段

- 能用构造函数建立类不变量。
- 能说明 RAII 如何处理提前返回和异常。
- 能根据需求选择 `vector`、`unordered_map`、`map` 等容器。
- 能把错误放进接口，而不是用魔法返回值。
- 能写出基本单元测试覆盖边界输入。

A.3 高级阶段

- 能用 `concept` 约束模板参数。
- 能判断 `ranges` 视图是否存在悬空风险。
- 能用测量定位性能瓶颈，而不是凭直觉优化。
- 能用 `mutex`、`atomic` 和 `condition variable` 解释同步边界。
- 能拆分大型类，控制依赖方向。

A.4 项目验收清单

- 每个公开函数都能说清楚所有权：观察、修改、保存副本还是接管资源。
- 每个错误路径都有测试或明确的人工验证步骤。
- 每个容器选择都能说出主要操作和复杂度理由。
- 每个类至少有一个围绕不变量的测试，而不只是 `getter` 测试。
- 每次性能优化都有优化前后的输入规模、构建模式和测量结果。
- 每个后台线程都有退出协议，程序关闭时不会依赖强制杀线程。

A.5 读代码自检问题

1. 这段代码有没有隐藏的资源释放规则？
2. 有没有返回局部对象的引用、保存短生命周期视图、或长期持有可能失效的迭代器？
3. 错误是被吞掉、打印后继续，还是进入了接口？
4. 测试是否覆盖空输入、单元素、边界值和失败路径？
5. 如果数据规模扩大一百倍，主要成本会落在哪里？

附录 B

标准库能力地图

领域	常用设施	学习重点
字符串	<code>std::string</code> , <code>std::string_view</code>	拥有和观察的区别
连续容器	<code>std::vector</code> , <code>std::array</code> , <code>std::span</code>	生命周期、扩容、边界
关联容器	<code>std::map</code> , <code>std::unordered_map</code>	有序性、哈希、复杂度
资源管理	<code>unique_ptr</code> , <code>shared_ptr</code> , RAII 类型	所有权表达
错误表达	<code>optional</code> , <code>error_code</code>	异常, 错误是否进入接口
算法	<code>find</code> , <code>sort</code> , <code>copy_if</code> , <code>accumulate</code>	用标准词汇表达意图
泛型 并发	<code>template</code> , <code>concept</code> , <code>type traits</code> <code>thread</code> , <code>mutex</code> , <code>atomic</code> , <code>condition_variable</code>	能力约束和实例化成本 共享状态和同步边界

这张表不是背诵列表。每学一个设施，都要问三件事：它是否拥有资源，它的复杂度合同是什么，它失效或出错时会怎样。

B.1 按问题选择设施

你遇到的问题	优先考虑	先问清楚
函数可能没有结果 函数要返回成功或错误	<code>std::optional</code> 结果结构、 <code>expected</code> 风格类型	没有结果是否还需要原因 调用者是否要按错误类型分支
只观察一段连续数据 只观察字符串内容 独占动态对象 共享生命周期对象	<code>std::span<const T></code> <code>std::string_view</code> <code>std::unique_ptr</code> <code>std::shared_ptr</code> , <code>std::weak_ptr</code>	底层数据是否活得足够久 是否会保存到调用之后 所有权在哪里转移 是否存在循环引用
按 key 快速查找	<code>std::unordered_map</code>	是否需要稳定顺序和自定义 hash
表达筛选转换 保护共享状态	<code>std::ranges</code> , <code>views</code> <code>std::mutex</code> , RAII 锁	视图是否可能悬空 锁保护的不变量是什么

B.2 容易误用的设施

- `std::string_view`: 它不拥有字符，不能替代需要长期保存的 `std::string`。
- `std::shared_ptr`: 它不是“不想思考所有权”的默认答案。
- `std::vector::reserve`: 它只预留容量，不创建元素。
- `std::remove`: 它不删除容器元素，需要配合 `erase`。
- ranges 视图: 它常常惰性且非拥有，返回视图前要检查底层生命周期。

- `std::atomic`: 它只解决特定共享变量的原子访问，不自动保护多个字段的不变量。

术语表

RAII Resource Acquisition Is Initialization。把资源生命周期绑定到对象生命周期。

不变量 对象在对外可见状态下必须始终成立的条件。

值语义 对象可以像普通值一样复制、赋值和组合，副本之间互不影响。

所有权 谁负责释放资源、谁能延长资源生命周期的规则。

悬空引用 引用或指针指向的对象已经销毁。

迭代器失效 容器操作后，原迭代器不再安全可用。

concept C++20 中用于约束模板参数能力的语言机制。

数据竞争 多个线程并发访问同一对象，至少一个写入，且没有同步。

视图 观察已有数据的轻量对象，通常不拥有底层数据。

严格弱序 排序比较器需要满足的关系，相等元素之间不能互相小于对方。

死锁 多个执行单元互相等待对方释放资源，导致都无法继续。

RAII 锁 用对象构造和析构管理加锁、解锁的同步写法。

索引

invariant, 20

不变量, 20